Université Mohamed Premier
Ecole Supérieure de Technologie de Nador

Département d'Innovation Technologique et Ingénierie Informatique

Filière d'Ingénierie Logicielle et Cybersécurité

Rapport de projet de fin d'études

Pour l'obtention du

Diplôme Universitaire de Technologie

Sous thème :

MyCampus
Plateforme académique de gestion universitaire

Réalisé par :

ALAE MAJJATI : 2407075
NORHANE RAMZI : 2416666
MOHAMED AYMANE JLIDA : 2416663

Encadré par :

Mr. ESBAI REDOUANE, Professeur à l'ESTN

Soutenu le 31/03/2026, devant le jury :

Mr. Redouane ESBAI, Professeur à l'ESTN
Mr. Fouad ELOTMANI, Professeur à l'ESTN

Année universitaire : 2025/2026

DÉDICACE

Nous avons l'honneur et le privilège de dédier ce rapport de Projet de Fin d'Études, réalisé dans le cadre de notre formation en Ingénierie Logicielle et Cybersécurité à l'École Supérieure de Technologie de Nador, à celles et ceux qui ont été les piliers indéfectibles de notre parcours académique et professionnel.

À nos chers parents, dont le soutien indéfectible, les sacrifices quotidiens et la confiance absolue ont constitué la force motrice de nos efforts. Leur amour inconditionnel et leurs prières ferventes ont illuminé les moments de doute et célébré chacune de nos réussites, nous rappelant constamment l'importance de la persévérance et de l'excellence.

À notre distingué encadrant, Monsieur ESBAI Redouane, enseignant-chercheur émérite, nous exprimons notre profonde gratitude pour son accompagnement rigoureux, ses conseils avisés et sa disponibilité exemplaire. Son expertise pointue en architectures full-stack et sa bienveillance pédagogique ont été déterminants pour orienter ce projet vers les standards d'excellence professionnelle.

À l'ensemble du corps professoral de notre filière, nous rendons hommage à leur engagement sans faille dans la transmission des connaissances fondamentales et avancées qui constituent aujourd'hui notre socle technique solide. Leur passion pour l'innovation et la rigueur scientifique a forgé notre esprit critique et notre aptitude à relever les défis technologiques contemporains.

À nos familles élargies et à nos amis proches, complices précieux dans les veilles studieuses, les échanges constructifs et le soutien moral constant. Leur encouragement permanent a été le catalyseur de notre détermination face aux complexités du développement de MyCampus.

À l'ensemble de nos collègues et camarades de promotion, avec qui nous avons partagé les joies des découvertes techniques et surmonté ensemble les obstacles inhérents à un projet d'envergure tel que la plateforme MyCampus.

Enfin, à Allah l'Unique, Le Tout Miséricordieux, Le Très Haut, Source de toute science et de toute réussite, nous adressons nos actions de grâce les plus sincères pour Ses innombrables bienfaits, Sa guidance constante et la grâce d'avoir mené à bien ce travail dans les meilleures conditions.

REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à l'ensemble des personnes et institutions qui ont contribué à la réalisation de ce Projet de Fin d'Études portant sur la conception et le développement d'une plateforme de gestion universitaire.

Tout d'abord, nous adressons nos sincères remerciements à notre encadrant, Monsieur ESBAI Redouane, enseignant-chercheur à l'École Supérieure de Technologie (EST) d'Oujda. Sa disponibilité exemplaire, ses conseils avisés et son expertise pointue en ingénierie logicielle et cybersécurité ont été indispensables pour orienter notre travail, résoudre les défis techniques et assurer la qualité de notre plateforme. Sans son accompagnement rigoureux et sa bienveillance constante, ce projet n'aurait pas atteint son niveau d'excellence.

Nous remercions également l'ensemble du corps professoral de la filière Ingénierie Logicielle et Cybersécurité de l'EST, pour la formation solide qu'ils nous ont dispensée tout au long de notre parcours. Un hommage particulier est rendu aux membres du jury de soutenance, dont la présence et l'expertise enrichiront nos échanges lors de la défense.

Nos pensées vont à nos familles et amis, dont le soutien moral inconditionnel a été une source de motivation inébranlable face aux défis rencontrés.

Enfin, nous exprimons notre reconnaissance à l'Université Mohammed Premier d'Oujda et à l'Ecole Supérieure de Technologie de Nador pour avoir mis à notre disposition les ressources nécessaires, tant matérielles qu'humaines, à la réalisation de ce projet.

Ce PFE, réalisé par ALAE MAJJATI, NORHANE RAMZI et MOHAMED AYMANE JLIDA, représente le fruit d'un travail collectif et engagé. Nous restons ouverts à toute suggestion d'amélioration.

Résumé

MyCampus, est une plateforme académique intelligente accessible via web et mobile. Cette solution digitalise la gestion d'un établissement d'enseignement supérieur en remplaçant les processus manuels obsolètes comme Excel, les e-mails et les documents papier. Elle centralise les données et workflows autour de trois rôles principaux : administrateur, professeur et étudiant, avec un contrôle d'accès granulaire assuré par RBAC (Role-Based Access Control).

Sur le plan fonctionnel, MyCampus modélise la structure académique incluant départements, filières, classes, modules et matières, tout en gérant l'affectation des enseignements. Elle intègre la gestion des utilisateurs avec création, édition et import massif d'étudiants via fichiers, ainsi que les services pédagogiques essentiels : séances, présences et absences avec justifications, examens, notes et relevés de résultats. Un module de communication repose sur des annonces et notifications push, complété par des tableaux de bord personnalisés pour chaque profil. Côté innovation, un chatbot frontend est connecté à un workflow n8n via webhook, suivant une approche RAG pour des interactions intelligentes.

Techniquement, l'architecture full-stack repose sur Spring Boot avec Java 17 pour le backend, qui expose une API REST documentée via Swagger et springdoc, enrichie de WebSocket pour les fonctionnalités temps réel. Le frontend web utilise React avec Vite, tandis que l'application mobile est développée en Flutter pour une compatibilité multiplateforme. La base de données MySQL stocke l'ensemble des données, et le projet est entièrement dockerisé via Docker Compose, incluant les services MySQL, backend, frontend, phpMyAdmin et n8n, garantissant portabilité, scalabilité et déploiement aisé.

MyCampus optimise l'efficacité administrative, améliore la traçabilité pédagogique et renforce la cybersécurité. Ce projet démontre une maîtrise des stacks full-stack, de l'IA conversationnelle et des pratiques DevOps, avec un prototype fonctionnel prêt à l'emploi.

Table des matières

DÉDICACE 3

REMERCIEMENTS 4

Résumé 5

Table des matières 6

Liste des tableaux 16

Glossaire des acronymes 16

Introduction générale 17

Chapitre 1 : Étude de l'existant et analyse des besoins : 19

1 Étude de l'existant 20

1.1 Analyse du système actuel 20

1.1.1 Le système traditionnel (Processus manuel et physique) 20

1.1.2 La plateforme web existante (Limites techniques) 20

1.2 Limites et problèmes identifiés 21

1.2.1 L'opacité et l'absence de suivi en temps réel 21

1.2.2 La lourdeur des processus administratifs (Dépendance physique) 21

1.2.3 Fragmentation et dispersion de l'information 21

1.2.4 Manque d'interactivité et de services numériques modernes 22

1.2.5 Failles potentielles de sécurité et de confidentialité 22

2 Expression des besoins 22

2.1 Cahier des charges 22

2.1.1 Présentation du projet 22

2.1.2 Périmètre et acteurs 23

2.1.3 Exigences fonctionnelles (principales) 23

2.2 Méthodologie de collecte des besoins (Étude de terrain) 24

2.2.1 Analyse du taux de satisfaction 24
2.2.2 Identification des domaines de difficultés 25
2.2.3 Fonctionnalités les plus attendues 25
2.3 Besoins fonctionnels 26
2.3.1 Besoins pour l'Administrateur (Gestion globale) 26
2.3.2 Besoins pour le Professeur (Gestion pédagogique) 26
2.3.3 Besoins pour l'Étudiant (Utilisateur final) 27
2.4 Besoins non fonctionnels 27
2.5 Contraintes techniques 28
2.5.1 Contrainte de Déploiement et de Portabilité (Docker) 28
2.5.2 Contrainte de Sécurité des Données (RGPD & Confidentialité) 28
2.5.3 Contrainte de Réactivité et de Performance 28
2.5.4 Contrainte d'Ergonomie et d'Accessibilité (Responsivité) 28
2.5.5 Contrainte d'Intégration de l'IA (Chatbot) 29
2.5.6 Contrainte de Maintenance et d'Évolutivité 29
3 Spécifications fonctionnelles 30
3.1 Diagramme de cas d'utilisation 30
3.2 Description textuelle des cas 30
3.2.1 Cas d'utilisation : Gérer les Notes (Acteur : Enseignant) 30
3.2.2 Cas d'utilisation : Soumettre une justification d'absence (Acteur : Étudiant) 31
3.2.3 Cas d'utilisation : Gérer les utilisateurs et imports (Acteur : Administrateur) 32
Chapitre 2 : Conception du système : 33
1 Architecture globale 34
1.1 Choix de l'architecture 34
1.2 Justification des choix technologiques 35
2 Conception détaillée 37

2.1 Pourquoi utiliser l'UML ? 37
2.2 Outil utilisé pour la modélisation UML 37
2.3 Diagramme de classes 38
2.3.1 Description des modules présentés 38
2.3.2 Justification de la modularité 39
2.4 Diagramme de séquence 39
2.4.1 Phase d'authentification (Login) 40
2.4.2 Utilisation du JWT 40
2.4.3 Rafraîchissement du token (Refresh Token) 40
2.4.4 Déconnexion (Logout) 41
2.4.5 Aspects de sécurité 41
2.5 Diagramme de composants 42
2.5.1 Composants côté client 42
2.5.2 Composant Backend (Spring Boot) 43
2.5.3 Composant Base de données (MySQL) 43
2.5.4 Stockage de fichiers 44
2.5.5 Composant n8n (Chatbot RAG) 44
2.5.6 Service externe : reCAPTCHA 45
2.5.7 Flux global de communication 45
2.6 Diagramme de déploiement 45
2.6.1 Environnement Docker 46
2.6.2 Les conteneurs principaux 46
2.6.3 Gestion des volumes (persistance) 47
2.6.4 Réseau Docker 47
2.6.5 Flux de déploiement 47
2.6.6 Sécurité du déploiement 48
2.7 Diagramme d'activité 48

Chapitre 3 : Implémentation : 50
1 Environnement de développement 51
1.1 Outils et logiciels. 51
2 Technologies utilisées 51
2.1 Backend 51
2.2 Frontend Web 52
2.3 Mobile 52
3 Description des modules développés 52
3.1 Authentification et sécurité 52
3.2 Gestion de la structure académique (Admin) 53
3.3 Gestion des étudiants (Admin) 53
3.4 Pédagogie 53
3.5 Communication 53
3.6 Emploi du temps 53
4 Présentation de l'intégration n8n 54
4.1 Objectif du workflow 54
4.2 Comment ce RAG fonctionne ? 55
4.2.1 Déclenchement et entrée du workflow 55
4.2.2 Guardrail IN (filtrage avant LLM) 55
4.2.3 Logique de routage (If / If-Document) 55
4.2.4 Branche "Document" (PDF / Cours) 56
4.2.5 Branche "Non-document" (données MyCampus) 56
4.2.6 Règles d'accès et confidentialité (par rôle) 57
4.2.7 Guardrail OUT (filtrage après LLM) 57
5 Captures d'écran commentées 58
5.1 Gestion des annonces (Professeur) 58
5.2 Espace Étudiant : présences et justification 59

5.3 ChatBot (web) 60
Chapitre 4 : Sécurité : 61
1 Analyse des risques 62
2 Politique de sécurité 62
3 Mécanismes implémentés : 62
3.1 Authentification 62
3.2 Autorisation (RBAC) 63
3.3 Chiffrement / stockage sécurisé 63
3.4 Sécurisation WebSocket 63
3.5 Protection contre : 63
4 Tests de sécurité (OWASP ZAP) 64
4.1 Audit de sécurité avec OWASP ZAP 64
4.1.1 Méthodologie du scan 64
4.2 Objectif du test 65
4.3 Périmètre et environnement 65
4.4 Résumé des résultats 65
4.5 Détails des alertes détectées 65
4.5.1 Alertes critiques (High) 65
4.5.2 Alertes importantes (Medium) 66
4.5.3 Alertes faibles (Low) 66
4.5.4 Alertes informatives (Informational) 67
4.6 Synthèse des remédiations 68
Vulnérabilité 68
Risque 68
Correction Technique (Spring Security) 68
CSP Missing 68
Vol de session (XSS) 68

.contentSecurityPolicy(csp -> csp.policyDirectives(...)) 68
Clickjacking 68
Détournement de clic 68
.frameOptions(frame -> frame.deny()) 68
MIME Sniffing 68
Exécution de scripts 68
.contentTypeOptions(withDefaults()) 68
HSTS 68
Interception de trafic 68
.httpStrictTransportSecurity(hsts -> hsts.includeSubDomains(true)...) 68
Chapitre 5 : : Tests et validation : 69
1 Plan de test : Types de tests réalisés 70
1.1 Introduction 70
1.1.1 Tests unitaires 70
1.1.2 Tests des présences 72
1.1.3 Tests des examens et des notes 73
1.1.4 Tests des annonces 77
1.2 Résultats des tests 78
1.2.1 Conclusion 79
1.3 Les tests d'intégration 80
1.3.1 Test N°1 : Validation globale du module des examens 81
1.3.2 Test N°2 : Validation intégrée du module des notes 82
1.3.3 Test N°3 : Validation intégrée du module des présences 83
1.3.4 Test N°4 : Validation du module d'authentification et de sécurité 83
1.3.5 Conclusion 84
1.4 Tests de Performance 84
1.4.1 Configuration du Test Plan 85

1.4.2 Analyse des métriques de performance 86
1.4.3 Résultats et Interprétation 86
1.4.4 Conclusion des Tests de Performance 88
Conclusion et perspectives 89
Référence bibliographique 91
Annexes 91
1 Captures d''écran 92
1.1 La version mobile : 92
1.2 La version web : 94
2 Code source 95
2.1 Authentification + Refresh token (cookie HTTP-only) 95
2.2 Workflow n8n : Guardrail IN (anti-injection) + classification "document" 96
3 Diagramme de Gantt : 97
4 Manuel utilisateur – MyCampus (Web + Mobile) 97
4.1 Présentation 97
4.2 Accès à l'application 98
4.2.1 Version Web 98
4.2.2 API Backend (référence) 98
4.2.3 Base de données (admin) 98
4.2.4 n8n (chatbot/workflows) 98
4.3 Connexion / Déconnexion 98
4.3.1 Se connecter 98
4.3.2 Rafraîchir la session 98
4.3.3 Se déconnecter 98
4.4 Fonctionnalités communes (tous rôles) 98
4.4.1 Profil utilisateur 98
4.4.2 Notifications 99

4.4.3 Chatbot (Web) 99
4.5 Guide Étudiant 99
4.5.1 Tableau de bord Étudiant 99
4.5.2 Espace pédagogique 99
4.5.3 Présences / absences 99
4.5.4 Notes / relevé 100
4.5.5 Emploi du temps 100
4.5.6 Annonces 100
4.6 Guide Professeur 100
4.6.1 Tableau de bord Professeur 100
4.6.2 Séances / ressources 100
4.6.3 Présences 100
4.6.4 Notes / examens 100
4.6.5 Annonces Professeur 100
4.6.6 Emploi du temps 100
4.7 Guide Administrateur 101
4.7.1 Tableau de bord Administratif 101
4.7.2 Gestion de la structure académique 101
4.7.3 Affectations d'enseignement 101
4.7.4 Gestion des utilisateurs 101
4.7.5 Import des étudiants 101
4.7.6 Gestion des annonces (Admin) 101
4.7.7 Emploi du temps (Admin) 101
4.7.8 Sécurité RBAC 101
4.8 Bonnes pratiques et sécurité 101
4.9 Dépannage (problèmes fréquents) 102
5 Planification de projet : 102

5.1 Gestion de Projet avec la Méthode Agile (scrum) 102
5.1.1 Outils Utilisés 102
5.1.2 Processus Scrum Implémenté 103
 Planification de Sprint (Sprint Planning) 103
 Synchronisations Quotidiennes 103
 Sprint Reviews et Démonstrations 104
 Rétrospectives (Retours d'Expérience) 104
5.2 Estimation des taches 104
6 Lien GitHub du projet : 111

Liste des figures

Figure 1 : Diagramme analyse du taux de satisfaction 24
Figure 2 : Diagramme des domaines de difficultés 25
Figure 3 : Diagramme des fonctionnalités les plus attendues 26
Figure 4 : Diagramme de cas d'utilisation 30
Figure 5 : Logo de Spring boot 35
Figure 6 : Logo de REACT et VITE 35
Figure 7 : Logo de Flutter 35
Figure 8 : Logo de N8N 36
Figure 9 : Logo de MYSQL 36
Figure 10 : Logo de Docker 37
Figure 11 : logo d'outil de modélisation 37
Figure 12 : Extrait du diagramme de classe 38
Figure 13 : Diagramme de séquence 39
Figure 14 : Diagramme de composants 42
Figure 15 : Diagramme de déploiement 45
Figure 16 : Diagramme d'activité 49
Figure 17 : Logo des outils et logiciels. 51
Figure 18 : Workflow de n8n 54
Figure 19 : Page de création d'annonces pour Professeur 58
Figure 20 : Page de consultation da la présence d'étudiant 59
Figure 21 : Page de jusfication d'absence 59
Figure 22 : Page du chatbot 60
Figure 23 : Logo de Zap 64
Figure 24 : Resultat de scan par Zap 65
Figure 28 : Résultats des tests d'integration 79
Figure 29 : Les tests d'intégration 81
Figure 30 : Test unitaire N°1 82
Figure 31 : Configuration du Test Plan 85
Figure 32 : Analyse des métriques de performance 86
Figure 33 : Fiabilité et Stabilité 87

Liste des tableaux

Tableau 1 : 1er cas d'utilisation 31
Tableau 2 : 2eme cas d'utilisation 32
Tableau 3 : 3eme cas d'utilisation 32
Tableau 4 : Architecture de projet 35
Tableau 5 : Outils de développements 51
Tableau 7 : Tests de l'authentification et de la sécurité 72
Tableau 8 : Tests des présences 73
Tableau 9 : Tests des examens et des notes 76
Tableau 10 : Tests des annonces 78

Glossaire des acronymes

Acronyme	Signification
RBAC	Role-Based Access Control
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
UX	User Experience
UI	User Interface
UML	Unified Modeling Language
CRUD	Create, Read, Update, Delete
RAG	Retrieval Augmented Generation
PFE	Projet de Fin d'étude

Introduction générale

Dans un contexte de transformation digitale accélérée des établissements d'enseignement supérieur au Maroc, la gestion académique reste largement tributaire de procédures manuelles et dispersées, telles que l'utilisation de fichiers Excel, les échanges par e-mail et les documents papier. Cette dépendance génère des retards administratifs récurrents, un manque de visibilité en temps réel sur les données pédagogiques, une traçabilité insuffisante des opérations, et oblige les étudiants à des déplacements fréquents vers les services administratifs pour des tâches répétitives comme la consultation de notes ou la justification d'absences.

Le choix de ce sujet s'inscrit directement dans un besoin concret observé en tant qu'étudiants de l'École Supérieure de Technologie (EST) de Nador. Nous aspirons à une plateforme unique permettant de consulter les notes d'examen, de générer des relevés, de suivre les absences et de les justifier, d'accéder aux annonces et informations essentielles, sans recourir systématiquement aux démarches en présentiel. Au-delà des gains de temps et d'efficacité, cette solution doit impérativement sécuriser les informations personnelles et garantir la confidentialité des données académiques sensibles, dans un cadre respectueux des normes de cybersécurité.

Face à cette problématique, notre Projet de Fin d'Études (PFE) propose MyCampus, une plateforme académique intelligente multi-canaux (web et mobile) qui centralise et automatise les processus clés de la vie universitaire. Adaptée à trois rôles principaux – Administrateur, Professeur et Étudiant –, elle implémente un contrôle d'accès fin basé sur les rôles et permissions (RBAC), assurant une utilisation sécurisée et personnalisée pour chaque profil utilisateur. Ce projet répond à l'urgence de digitalisation dans les établissements comme l'EST, où les outils legacy freinent l'innovation pédagogique.

MyCampus s'appuie sur une architecture full-stack moderne comprenant un backend Spring Boot (Java 17) exposant des API REST documentées (Swagger/springdoc) et WebSocket, un frontend web en React avec Vite, une application mobile en Flutter, et une base de données MySQL. L'ensemble est containerisé via Docker Compose (incluant MySQL, backend, frontend, phpMyAdmin et n8n pour les workflows intelligents comme le chatbot

RAG), facilitant le déploiement, les tests et la scalabilité. Les défis identifiés incluent la fragmentation des données, les erreurs humaines dans les saisies manuelles, le manque d'interopérabilité et les risques sécuritaires, que MyCampus résout par une centralisation des workflows (structure académique, affectations, présences, notes, communications) et une intégration d'IA pour des interactions fluides.

Les objectifs généraux visent à développer une plateforme ergonomique, sécurisée et extensible, spécifiquement à modéliser la structure académique, automatiser la gestion pédagogique, implémenter RBAC et notifications, assurer une expérience multiplateforme, et démontrer via un prototype dockerisé. Nous avons suivi une approche agile (Scrum) avec itérations : analyse des besoins, conception UML/MERISE, développement full-stack, tests unitaires/intégration (Postman), et déploiement Docker, soutenus par des outils comme Git et Swagger.

Ce rapport présente d'abord le contexte et l'analyse des besoins, puis la conception fonctionnelle et technique, l'implémentation détaillée, les tests et résultats, avant de conclure sur les perspectives d'évolution

Chapitre 1 : Étude de l'existant et analyse des besoins :

1 Étude de l’existant

1.1 Analyse du système actuel

L'analyse du fonctionnement de la scolarité à l'EST de Nador révèle un système en transition, mais qui souffre encore de lourdeurs administratives et technologiques majeures. Le processus actuel peut être divisé en deux méthodes :

1.1.1 Le système traditionnel (Processus manuel et physique)

Jusqu'à présent, la gestion quotidienne repose sur des méthodes classiques qui posent des problèmes de réactivité :

- **Gestion des notes et affichage :**

Les notes sont saisies sur des fichiers Excel par les professeurs, puis transmises à l'administration. L'étudiant ne prend connaissance de ses résultats que via des listes d'affichage physiques placardées dans l'école ou envoyées de manière informelle.

- **Gestion des absences :**

Pour justifier une absence, l'étudiant doit se déplacer physiquement au service de scolarité pour déposer des documents papier. Ce processus manuel est source d'erreurs, de pertes de documents et de retards de traitement.

- **Documents académiques :**

Toute demande de relevé de notes ou d'attestation nécessite une présence physique et un délai de traitement administratif, car le système n'est pas automatisé pour la génération instantanée de documents.

1.1.2 La plateforme web existante (Limites techniques)

Une plateforme en ligne a été récemment introduite pour moderniser ces échanges, mais elle présente des lacunes critiques qui limitent son utilité :

- **Accès tardif aux informations :**

Le plus grand défaut de ce site est la temporalité. Les notes ne sont publiées qu'à la toute fin du semestre ou de l'année. L'étudiant n'a aucune visibilité sur son contrôle continu, ce qui l'empêche de s'évaluer et de progresser durant le semestre.

- **Manque de fonctionnalités :**

La plateforme se limite à une consultation passive. Elle ne permet pas de soumettre des justificatifs d'absence, de contacter les professeurs, ou d'accéder à des ressources pédagogiques en temps réel.

- **Organisation et Ergonomie :**

Le site souffre d'un manque d'organisation dans la présentation des données, rendant l'expérience utilisateur frustrante et peu intuitive.

1.2 Limites et problèmes identifiés

L'examen du système actuel (processus manuels et plateforme web existante) met en évidence plusieurs dysfonctionnements critiques qui impactent négativement la vie académique des étudiants et l'efficacité de l'administration :

1.2.1 L'opacité et l'absence de suivi en temps réel

C'est le problème majeur ressenti par les étudiants. L'actuelle plateforme ne sert que d'outil d'archivage final.

- **Conséquence** : L'étudiant ne connaît ses notes qu'une fois le semestre terminé. Il ne peut pas identifier ses lacunes à temps pour se rattraper, ce qui va à l'encontre du principe de l'évaluation continue.

1.2.2 La lourdeur des processus administratifs (Dépendance physique)

Le système repose encore trop sur le "présentiel" pour des tâches qui devraient être automatisées.

- **La gestion des absences** : Devoir se déplacer pour déposer un justificatif papier est une perte de temps pour l'étudiant et une source d'encombrement pour les bureaux de la scolarité.
- **La demande de documents** : Le processus de retrait d'attestations ou de relevés est lent et nécessite plusieurs allers-retours physiques.

1.2.3 Fragmentation et dispersion de l'information

L'information n'est pas centralisée sur un support unique et fiable :

- Les emplois du temps sont sur des panneaux ou des groupes WhatsApp.
- Les annonces officielles sont parfois ratées par manque de système de notification.
- Les notes sont sur Excel (côté prof) ou sur le site (en fin d'année).
- **Risque** : Cette fragmentation entraîne souvent une confusion chez l'étudiant et augmente le risque d'erreur de saisie ou de perte de données côté administration.

1.2.4 Manque d'interactivité et de services numériques modernes

Le système actuel ne propose aucune fonctionnalité "intelligente" ou interactive :

- **Absence de communication directe** : Aucun canal officiel n'existe sur la plateforme pour échanger avec les professeurs.
- **Inaccessibilité mobile** : Le site actuel n'est pas optimisé pour les smartphones, alors que c'est l'outil principal des étudiants.
- **Absence d'assistance** : Les questions fréquentes des étudiants (procédures, dates, documents) saturent les guichets car il n'existe pas d'agent conversationnel (Chatbot) pour y répondre 24h/24.

1.2.5 Failles potentielles de sécurité et de confidentialité

L'affichage des notes sur des listes physiques ou l'envoi de fichiers non sécurisés pose un problème de confidentialité. Chaque étudiant devrait avoir accès uniquement à ses propres données via un espace sécurisé et authentifié.

2 Expression des besoins

2.1 Cahier des charges

2.1.1 Présentation du projet

- **Intitulé** : MyCampus – Plateforme académique intelligente (Web + Mobile)
- **Contexte** : Projet de Fin d'Études visant à digitaliser la gestion académique d'un établissement (EST/école).
- **Objectif général** : Centraliser les services universitaires (suivi pédagogique, administration, communication) dans une plateforme unique, sécurisée et multi-rôles.

2.1.2 Périmètre et acteurs

❖ **Acteurs :**

- **Administrateur** : gestion structure académique, utilisateurs, annonces, sécurité.
- **Professeur** : gestion séances, présences, notes/examens, annonces.
- **Étudiant** : consultation des informations personnelles et académiques, justificatifs, annonces, notifications, emploi du temps.

❖ **Périmètre fonctionnel couvert :**

- Authentification et gestion de session (JWT + refresh token).
- RBAC (rôles/permissions) et administration RBAC.
- Gestion structure : départements, filières, classes, modules, matières.
- Affectation d'enseignement (professeur–classe–matière).
- Gestion des présences et justificatifs (upload).
- Gestion des notes/examens : feuille, saisie, publication, copies corrigées.
- Annonces (admin, professeur) et consultation étudiant.
- Notifications (liste, non lues, marquer lues).
- Emploi du temps (lecture JSON + prochaine séance).
- Gestion fichiers (photos, ressources, justificatifs, copies) + téléchargement.
- Chatbot web connecté à n8n (webhook).

2.1.3 Exigences fonctionnelles (principales)

❖ **Authentification / Sécurité**

- Connexion utilisateur avec reCAPTCHA : POST /auth/connecter.
- Rafraîchissement de session : POST /auth/rafraichir-jeton.
- Déconnexion : POST /auth/deconnexion.
- Profil connecté : GET /auth/me.
- RBAC appliqué sur les endpoints par permissions (@PreAuthorize).
- Administration RBAC : /api/admin/securite (CRUD + affectations).

❖ **Espace Administrateur**

- Gérer la structure académique (CRUD + ouvrir/fermer)
- Gérer les annonces
- Importer des étudiants par fichier : (CSV/XLS/XLSX, mode PARTIEL/ATOMIC).
- Consulter l'emploi du temps par classe

❖ **Espace Professeur**

- Gérer les présences d'une séance
- Gérer les notes : feuille, saisie, publication
- Déposer et télécharger des copies corrigées
- Gérer les annonces

❖ **Espace Étudiant**

- Consulter modules/matières/séances/ressources
- Consulter présences/absences
- Justifier une absence
- Consulter relevé par semestre
- Consulter annonces
- Consulter emploi du temps + prochaine séance

2.2 Méthodologie de collecte des besoins (Étude de terrain)

Pour valider la pertinence de notre projet et définir les fonctionnalités prioritaires, nous avons mené une étude quantitative auprès des étudiants de l'EST de Nador via un formulaire Google Forms. Voici l'analyse des résultats obtenus :

2.2.1 Analyse du taux de satisfaction

Le premier constat est sans appel : une large majorité des étudiants exprime un mécontentement vis-à-vis des outils actuels.

- **Résultats** : Environ 83,3% des répondants se disent "Insatisfaits" du système actuel.
- **Interprétation** : Cela confirme que les solutions existantes (affichage physique et plateforme limitée) ne répondent plus aux attentes des utilisateurs modernes.

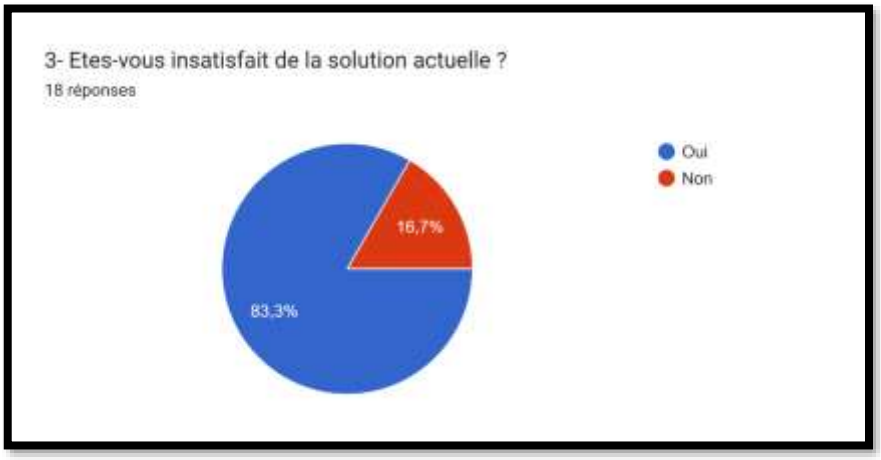


Figure 1 : Diagramme analyse du taux de satisfaction

2.2.2 Identification des domaines de difficultés

Afin de cibler les modules prioritaires de MyCampus, nous avons demandé aux étudiants d'identifier les domaines où ils rencontrent le plus de difficultés avec le système actuel.

- **Résultats** : Les réponses montrent une accumulation de problèmes sur plusieurs services clés :
  - 25% des difficultés concernent la consultation des Notes.
  - 21,9% concernent l'accès aux Emplois du temps.
  - 21,9% concernent la gestion des Absences.
  - 18,8% concernent la réception des Annonces officielles.
- **Interprétation** : Ce graphique prouve que le mécontentement est généralisé. Il justifie la nécessité d'une plateforme multiservices. MyCampus ne doit pas être un simple site de notes, mais un véritable écosystème regroupant ces quatre piliers pour supprimer ces points de friction.

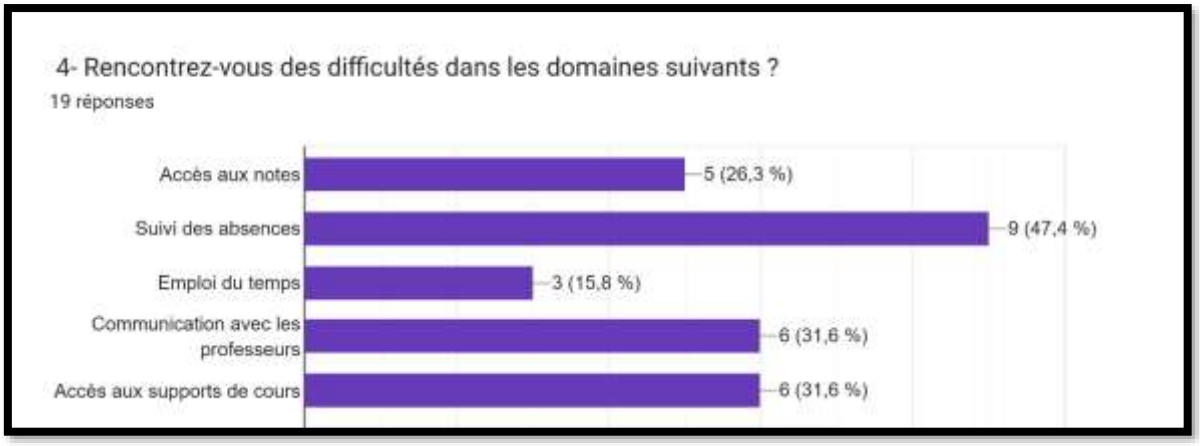


Figure 2 : Diagramme des domaines de difficultés

2.2.3 Fonctionnalités les plus attendues

Le sondage nous a permis de hiérarchiser les modules à développer en priorité.

- **Résultats** : La consultation des Notes et de l'Emploi du temps arrive en tête, suivie de près par la gestion des Absences et l'accès aux Annonces.
- **Interprétation** : Ces résultats ont directement dicté le backlog de notre projet et l'organisation de nos Sprints Scrum.

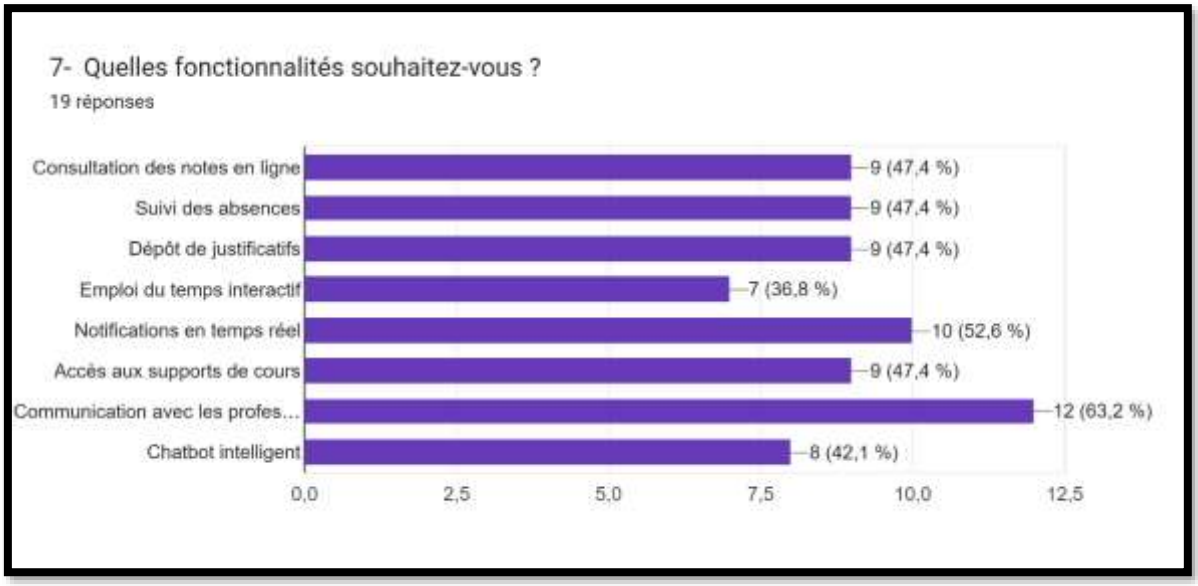


Figure 3 : Diagramme des fonctionnalités les plus attendues

2.3 Besoins fonctionnels

Suite à l'analyse des résultats de notre enquête (Google Forms) et des échanges avec les professeurs, nous avons identifié les besoins fonctionnels indispensables. Ces besoins sont segmentés selon les trois acteurs principaux de la plateforme MyCampus.

2.3.1 Besoins pour l'Administrateur (Gestion globale)

L'administrateur est le garant de la cohérence des données de l'établissement. Ses besoins incluent :

- **Gestion des utilisateurs** : Créer, modifier, suspendre ou supprimer des comptes (Étudiants, Professeurs).
- **Importation massive de données** : Pouvoir importer des listes d'étudiants ou de notes via des fichiers CSV/Excel pour éviter la saisie manuelle fastidieuse.
- **Gestion des structures** : Configurer les classes, les filières et les modules de l'ESTN.
- **Diffusion d'annonces** : Publier des communiqués officiels visibles par l'ensemble de l'école ou par une classe spécifique.
- **Supervision du système** : Accès à un tableau de bord récapitulatif des activités de la plateforme.

2.3.2 Besoins pour le Professeur (Gestion pédagogique)

Le professeur doit pouvoir gérer ses modules en toute autonomie :

- **Gestion des notes** : Saisir et modifier les notes des contrôles continus et des examens pour ses matières respectives.
- **Suivi des présences** : Marquer les absences lors des séances de cours et consulter l'historique d'un étudiant.
- **Partage de ressources** : Uploader des supports de cours (PDF, liens, exercices) accessibles directement par les étudiants.
- **Validation des justificatifs** : Consulter les justifications d'absences déposées par les étudiants pour les valider ou les refuser.

2.3.3 Besoins pour l'Étudiant (Utilisateur final)

C'est le cœur du système. L'étudiant doit bénéficier d'une transparence totale :

- **Tableau de bord personnel** : Consulter ses notes en temps réel dès qu'elles sont saisies par le professeur.
- **Gestion des absences** : Soumettre une justification d'absence de manière dématérialisée (upload de scan/photo du certificat médical).
- **Consultation de l'emploi du temps** : Accéder à son planning hebdomadaire mis à jour dynamiquement.
- **Accès aux annonces** : Recevoir les notifications et consulter le flux d'actualités de l'école.
- **Assistant intelligent (Chatbot)** : Interagir avec une IA pour obtenir des réponses rapides sur les procédures administratives, les dates d'examens ou les contacts utiles.

2.4 Besoins non fonctionnels

La solution doit répondre aux exigences suivantes :

- **Sécurité** : contrôle d'accès RBAC, protection des données personnelles, traçabilité des actions.
- **Disponibilité** : accès régulier et stable aux services, surtout en périodes d'examens.
- **Performance** : temps de réponse acceptable pour la consultation et le téléchargement (relevés, fichiers).
- **Fiabilité** : cohérence des données (notes/absences), réduction des erreurs de traitement.
- **Ergonomie** : interface simple, claire et accessible (web + mobile).

- Maintenabilité et évolutivité : architecture en couches, API REST, facilité d'ajout de fonctionnalités.

2.5 Contraintes techniques

Le développement de la plateforme MyCampus est soumis à plusieurs contraintes techniques majeures qui ont guidé nos choix technologiques et méthodologiques :

2.5.1 Contrainte de Déploiement et de Portabilité (Docker)

L'application doit pouvoir être déployée rapidement sur n'importe quel serveur sans conflit de version (entre la base de données, le backend et le frontend).

- **Solution** : L'utilisation de Docker et de Docker Compose est une contrainte imposée pour garantir que l'environnement de développement soit identique à l'environnement de production.

2.5.2 Contrainte de Sécurité des Données (RGPD & Confidentialité)

S'agissant de données académiques (notes, absences), la sécurité est une contrainte absolue.

- **Authentification** : Mise en place d'un système de jetons JWT (JSON Web Token) pour sécuriser les API.
- **Contrôle d'accès** : Implémentation du modèle RBAC (Role-Based Access Control) pour restreindre l'accès aux fonctionnalités selon le profil (Admin, Prof, Étudiant).
- **Validation** : Utilisation de mécanismes anti-robot comme reCAPTCHA sur les formulaires sensibles.

2.5.3 Contrainte de Réactivité et de Performance

La plateforme doit supporter des pics de connexion (par exemple, lors de la publication des notes) et répondre en moins de 2 secondes.

- **Solution** : Utilisation de technologies non bloquantes et performantes comme Spring Boot (Java) pour le backend, et une base de données relationnelle robuste (MySQL).

2.5.4 Contrainte d'Ergonomie et d'Accessibilité (Responsivité)

La majorité des étudiants accède à l'information via smartphone.

- **Solution** : L'interface doit impérativement être Responsive Design. Nous avons utilisé des frameworks CSS modernes (comme CSS et Bootstrap) pour assurer une compatibilité sur tous les terminaux (Mobiles, Tablettes, PC).

2.5.5 Contrainte d'Intégration de l'IA (Chatbot)

Le chatbot doit être capable de répondre de manière pertinente sans nécessiter une puissance de calcul colossale sur le serveur local.

- **Solution** : Utilisation de l'architecture RAG (Retrieval-Augmented Generation) via des outils comme n8n pour connecter l'IA aux documents de l'école de manière sécurisée et efficace.

2.5.6 Contrainte de Maintenance et d'Évolutivité

Le code doit être propre et documenté pour permettre à d'autres développeurs de reprendre le projet.

- **Solution** : Respect des principes SOLID et utilisation de l'architecture MVC (Modèle-Vue-Contrôleur) ou Clean Architecture.

3 Spécifications fonctionnelles

3.1 Diagramme de cas d'utilisation

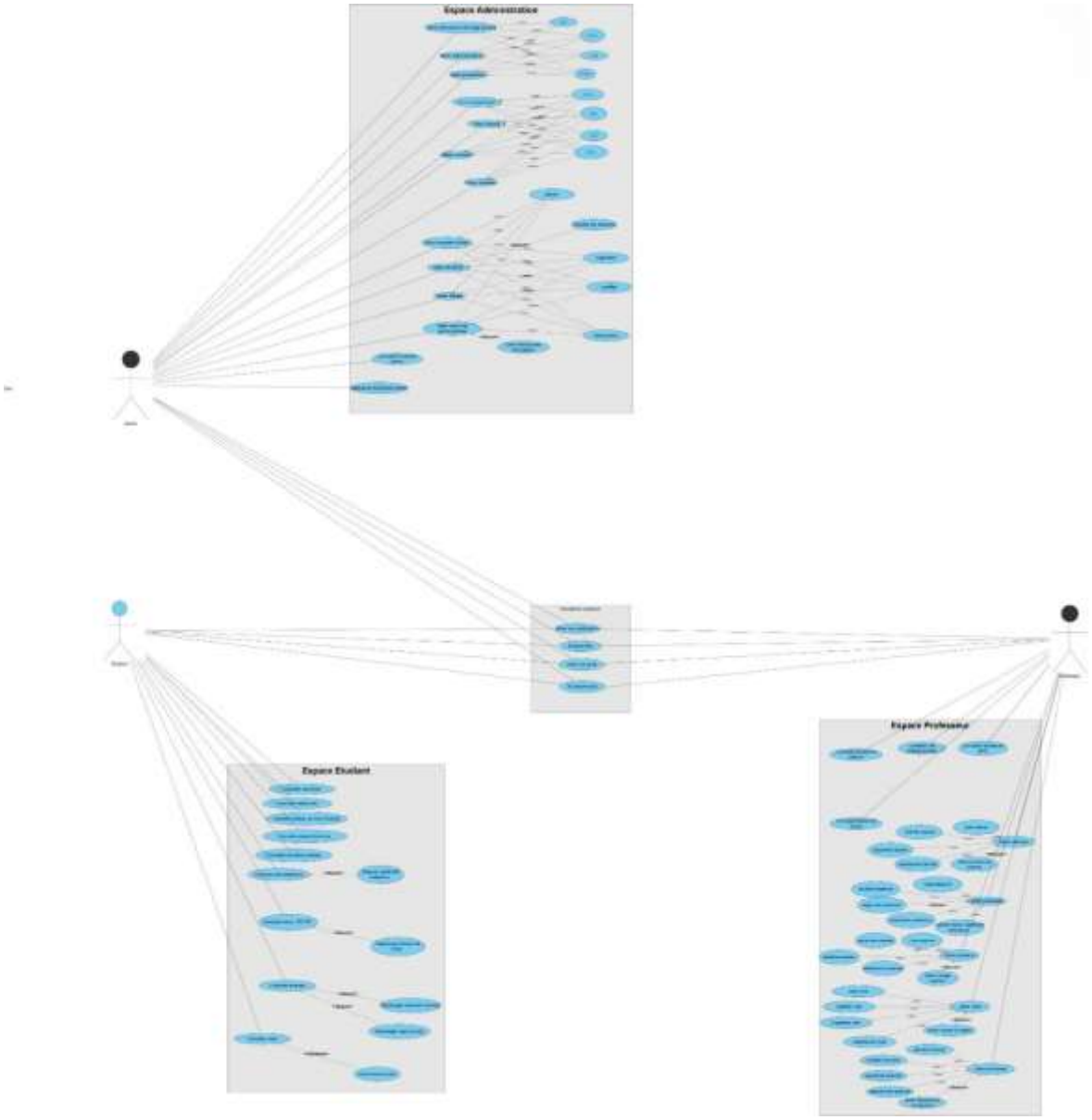


Figure 4 : Diagramme de cas d'utilisation

3.2 Description textuelle des cas

3.2.1 Cas d'utilisation : Gérer les Notes (Acteur : Enseignant)

Ce cas est crucial car il résout le problème de l'accès tardif aux résultats identifié dans votre sondage.

Élément	Description
Nom du cas	Gérer les notes (Ajouter/Modifier)
Acteur principal	Enseignant
Objectif	Permettre à l'enseignant de saisir les notes d'un contrôle ou d'un examen pour une classe spécifique.
Pré-conditions	L'enseignant doit être connecté et affecté à la matière et à la classe concernée.
Scénario nominal	• L'enseignant sélectionne "Mes Modules" puis choisit une classe. • Il clique sur "Gérer les notes" pour un examen donné. • Le système affiche la liste des étudiants inscrits. • L'enseignant saisit les notes dans les champs correspondants. • L'enseignant clique sur "Enregistrer". • Le système valide les données et les rend immédiatement visibles pour les étudiants.
Exceptions	Si une note est > 20 ou < 0 : le système affiche une alerte et empêche l'enregistrement.

Tableau 1 : 1er cas d'utilisation

3.2.2 Cas d'utilisation : Soumettre une justification d'absence (Acteur : Étudiant)

Ce cas répond au besoin de dématérialisation des processus administratifs de l'école.

Élément	Description
Nom du cas	Justifier une absence
Acteur principal	Étudiant
Objectif	Envoyer une preuve numérique (certificat médical, etc.) pour justifier une absence.
Scénario nominal	• L'étudiant se rend dans l'onglet "Mes Absences". • Il sélectionne l'absence non justifiée dans la liste. • Il clique sur "Joindre un justificatif".

	• Il télécharge un fichier (image ou PDF) et ajoute un commentaire. • Il valide l'envoi. • Le système notifie le professeur ou l'administrateur pour examen.
Exceptions	Si le format du fichier n'est pas autorisé (ex: .exe) : le système rejette le fichier.

Tableau 2 : 2eme cas d'utilisation

3.2.3 Cas d'utilisation : Gérer les utilisateurs et imports (Acteur : Administrateur)

C'est la partie "Back-office" qui assure le fonctionnement global de la plateforme.

Élément	**Description**
Nom du cas	Importer des données massives (CSV/Excel)
Acteur principal	Administrateur
Objectif	Ajouter rapidement une liste d'étudiants ou de professeurs sans saisie manuelle.
Scénario nominal	• L'administrateur accède au module "Gestion des utilisateurs". • Il choisit l'option "Importation groupée". • Il sélectionne le fichier CSV ou Excel préparé selon le modèle requis. • Le système analyse le fichier et affiche un aperçu des données. • L'administrateur confirme l'importation. • Le système crée automatiquement les comptes et génère les accès.
Exceptions	Si le fichier contient des emails déjà existants : le système signale les doublons et demande une correction.

Tableau 3 : 3eme cas d'utilisation

Chapitre 2 : Conception du système :

1 Architecture globale

1.1 Choix de l'architecture

MyCampus est construit selon une architecture monolithique distribué avec un modèle client-serveur stratifié, adaptée aux exigences d'un projet académique de fin d'études. Cette approche offre un équilibre optimal entre simplicité, maintenabilité et performance.

Elle repose sur une architecture trois tiers classique, décomposée en une couche de présentation, une couche métier et une couche donnée. Cette séparation garantit l'indépendance des composants et facilite la maintenance et l'évolution du système.

La couche présentation est assurée par deux clients distincts : une interface web Single Page Application (SPA) développée en React.js et une application mobile multiplateforme développée en Flutter. Ces deux clients communiquent exclusivement avec le backend via des requêtes HTTP REST.

La couche métier est portée par un serveur Spring Boot qui expose une API REST documentée via Swagger. Ce serveur implémente toute la logique fonctionnelle (gestion académique, sécurité, notifications, gestion de fichiers). Pour les communications temps réel, un serveur WebSocket est intégré au même processus Spring Boot.

La couche donnée est gérée par une base de données relationnelle MySQL 8.0, accédée via Spring Data JPA/Hibernate. Les fichiers uploadés (supports de cours, photos de profil, justifications) sont stockés sur le système de fichiers local dans un volume Docker persistant.

Couche	Technologie	Rôle
Présentation Web	React.js + Vite	Interface administrateur, professeur, étudiant
Présentation Mobile	Flutter (Dart)	Application étudiant et professeur (Android/iOS)
API REST	Spring Boot 4.0.1 (Java)	Logique métier, authentification, endpoints
WebSocket	Spring WebSocket	Notifications temps réel
Persistance	Spring Data JPA + Hibernate	ORM, accès base de données
Base de données	MySQL 8.0	Stockage relationnel
Fichiers	Système de fichiers local (Docker volume)	Cours, photos, justifications
Conteneurisation	Docker + Docker Compose	Déploiement unifié
Workflow Automation	N8N + RAG	chatbot intelligent RAG

Tableau 4 : Architecture de projet

1.2 Justification des choix technologiques

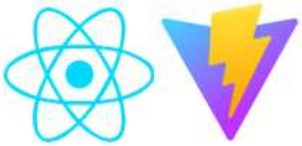
- **Backend**

Le backend repose sur Spring Boot avec Java 17, choix justifié par plusieurs avantages décisifs. Spring Boot est un framework mature et robuste, bénéficiant d'un écosystème extrêmement riche (Spring Data JPA, Spring Security, etc.). Il s'agit du standard de facto pour les APIs REST en entreprise, garantissant une qualité de code production-ready. L'intégration native d'Hibernate (ORM) simplifie la gestion des entités académiques complexes. De plus, Spring Boot facilite naturellement l'implémentation de patterns architecturaux modernes comme RBAC (contrôle d'accès basé sur les rôles) et les DTOs (Data Transfer Objects), essentiels au projet. La documentation automatique via Swagger (springdoc-openapi) assure une maintenance collaborative fluide.



Figure 5 : Logo de Spring boot

- **Frontend Web**



Le frontend web utilise React avec Vite, un choix moderne et performant. Vite révolutionne l'expérience de développement avec son Hot Module Replacement (HMR) ultra-rapide, réduisant considérablement le cycle de feedback développeur. React, grâce à son système de composants réutilisables, permet une scalabilité UI efficace. Cette stack moderne facilite l'intégration du chatbot n8n (RAG) via webhooks, apportant une dimension intelligente à la plateforme. La séparation claire Frontend/Backend via APIs REST assure une maintenabilité long terme.

Figure 6 : Logo de REACT et VITE

- **Frontend mobile**

Flutter a été sélectionné pour le développement mobile pour plusieurs raisons stratégiques. D'abord, c'est une technologie cross-platform, permettant de générer une application iOS et Android depuis une base de code unique en Dart. Cela réduit considérablement les coûts de développement comparé à du développement natif. Ensuite, Flutter offre des performances



Figure 7 : Logo de Flutter

quasi-natives tout en maintenant une excellente expérience utilisateur. Enfin, l'accès aux mêmes APIs REST du backend garantit une cohérence fonctionnelle et une UI/UX harmonisée sur toutes les plateformes.

- **N8N avec RAG**

N8N a été sélectionné comme moteur IA du projet pour implémenter un chatbot intelligent RAG. RAG (Retrieval Augmented Generation) est un paradigme puissant qui combine la recherche de documents pertinents avec la génération de réponses contextualisées. Contrairement aux modèles LLM classiques basés sur la mémorisation, RAG permet au chatbot MyCampus de répondre avec précision en s'appuyant sur les documents académiques réels (notes de cours, règlements, calendrier, etc.) stockés dans la plateforme.



Figure 8 : Logo de N8N

- **Base de données**

MySQL 8 a été choisi comme base de données pour ses qualités éprouvées. C'est un SGBD stable, fiable et gratuit/open-source. MySQL supporte de manière optimale les relations complexes inhérentes au modèle académique (Département → Filière → Classe → Modules → Matières → Affectations). La compatibilité native avec JPA/Hibernate élimine les frictions d'intégration ORM. De plus, MySQL fonctionne parfaitement dans Docker, aligné avec la stratégie de containerisation du projet. Les volumes de données académiques restent dans les capacités aisées de MySQL, sans nécessiter une base distribuée ou NoSQL.



Figure 9 : Logo de MYSQL

- **Déploiement**

Docker et Docker Compose assurent l'uniformité de l'environnement entre développement et production. Une seule commande (docker compose up --build) démarre l'ensemble complet : MySQL, Backend Spring Boot, Frontend React, et phpMyAdmin. Cela élimine les problèmes classiques "ça marche sur ma machine". L'isolation des services (base de données, backend, frontend dans des conteneurs séparés) améliore la sécurité et la gestion des ressources. Le scaling horizontal devient possible à moindre coût grâce à l'orchestration Docker.



Figure 10 : Logo de Docker

2 Conception détaillée

2.1 Pourquoi utiliser l'UML ?

L'UML permet une communication sans ambiguïté entre les différents acteurs du projet : développeurs, architectes, clients, et encadrants. En utilisant des diagrammes de classes, des diagrammes de cas d'usage et des diagrammes de séquence, chacun peut visualiser rapidement la structure et le comportement du système sans besoin d'interpréter du code.

2.2 Outil utilisé pour la modélisation UML

Pour la réalisation des différents diagrammes UML, j'ai utilisé l'outil Draw.io (diagrams.net). C'est une solution gratuite, accessible en ligne et très intuitive, qui permet de créer facilement des diagrammes professionnels (cas d'utilisation, classes, séquence, activité, etc.). L'avantage principal de Draw.io réside dans sa simplicité d'utilisation et sa large bibliothèque de formes adaptées aux standards UML.



Figure 11 : logo d'outil de modélisation

2.3 Diagramme de classes

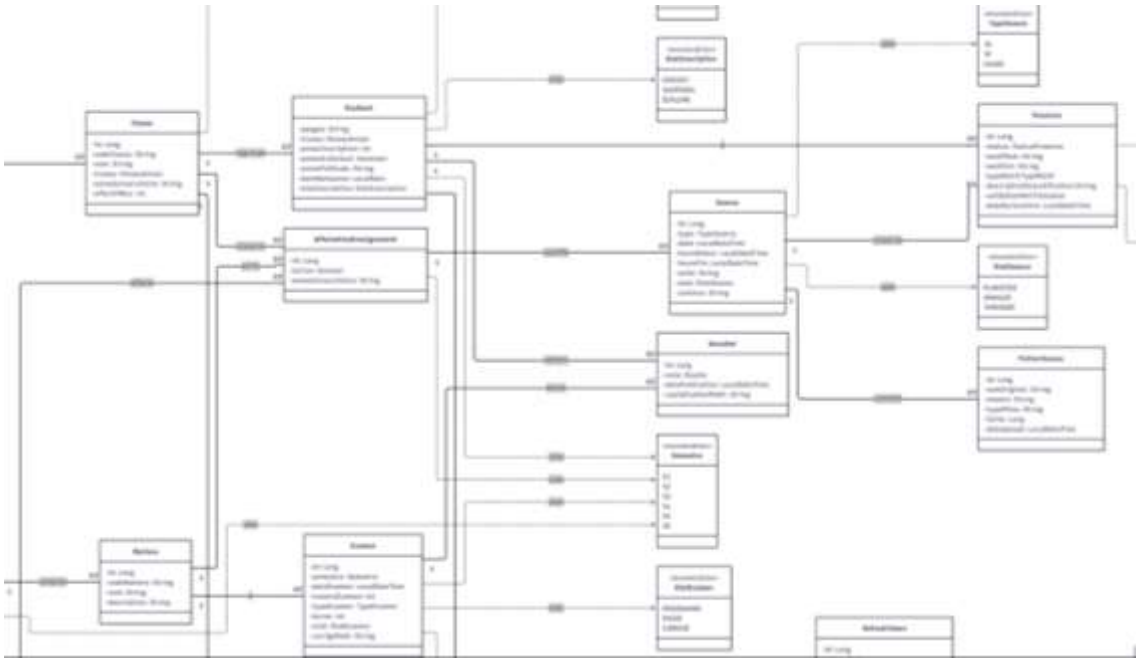


Figure 12 : Extrait du diagramme de classe

Le système MyCampus reposant sur une architecture complexe, nous présentons ici un extrait représentatif de notre diagramme de classes. Cette vue se concentre sur les entités piliers qui répondent aux problématiques majeures identifiées lors de notre étude de terrain (gestion des notes, des absences et de la structure académique).

2.3.1 Description des modules présentés

Bien que le système complet comporte d'autres classes techniques, cette partie du diagramme met en évidence trois blocs logiques essentiels :

- **Le Bloc Identité et Sécurité (User & Roles)** : Ce module gère l'authentification et l'autorisation. L'utilisation de l'héritage permet de distinguer les profils (Student, Teacher, Admin) tout en centralisant les attributs communs (nom, email, mot de passe).
- **Le Bloc Structure Académique (Department, Class, Course)** : Ces classes définissent l'organisation de l'EST de Nador. Elles permettent de lier un étudiant à une filière précise et d'affecter des enseignants à des modules spécifiques.
- **Le Bloc Opérationnel (Grade & Attendance)** : C'est le cœur métier de l'application.

- La classe Grade (Note) permet de stocker les résultats en temps réel.
- La classe Attendance (Absence) gère le suivi des présences et stocke le lien vers le fichier de justification déposé par l'étudiant.

2.3.2 Justification de la modularité

Cette partie du diagramme illustre la forte cohérence de notre base de données. Chaque entité est reliée par des associations précises, garantissant qu'aucune donnée ne soit isolée ou redondante.

2.4 Diagramme de séquence

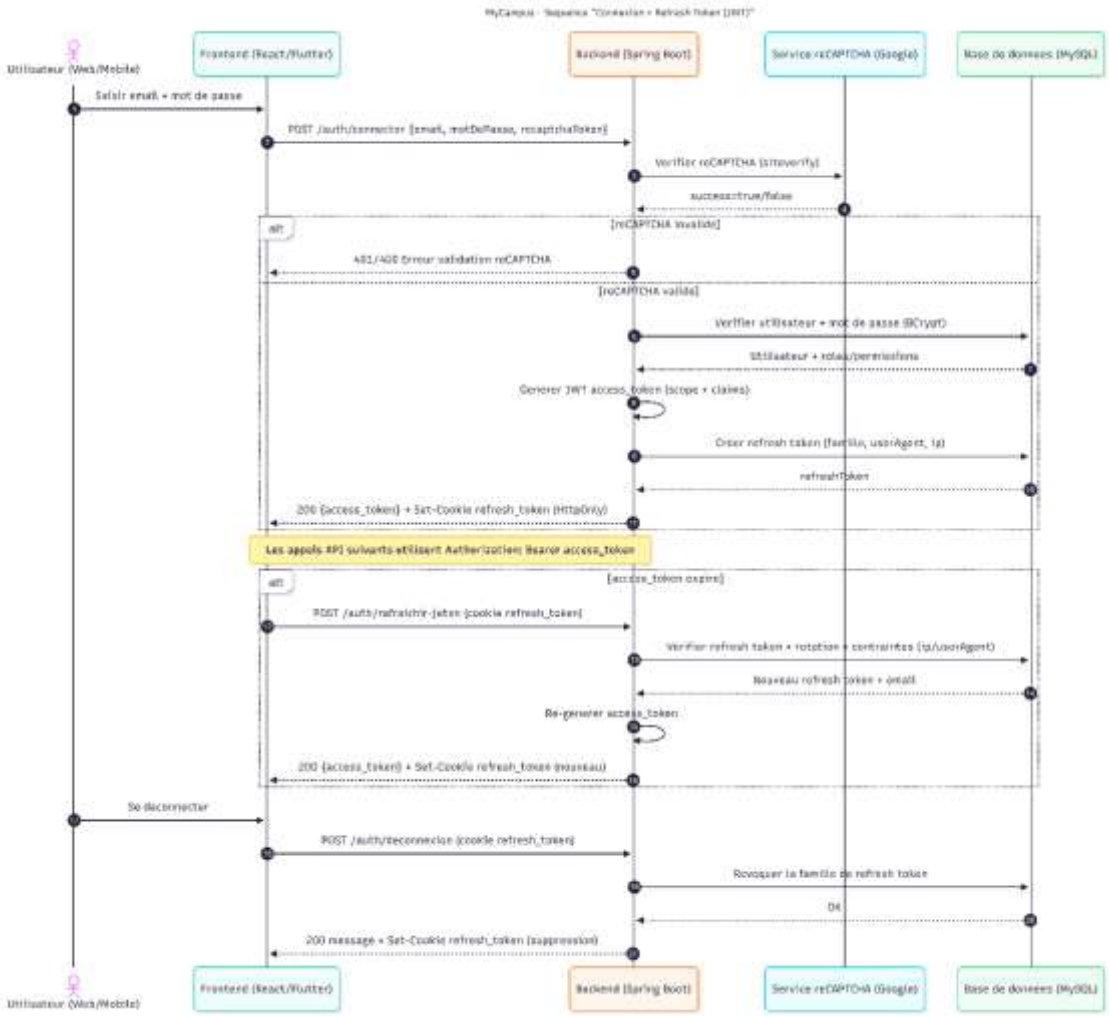


Figure 13 : Diagramme de séquence

Ce diagramme de séquence illustre le processus complet d'authentification et de gestion de session dans la plateforme MyCampus, en utilisant des mécanismes sécurisés basés sur JWT (JSON Web Token) et Refresh Token.

2.4.1 Phase d'authentification (Login)

Le processus débute lorsque l'utilisateur (via une interface web ou mobile) saisit son email et son mot de passe. Le frontend envoie ensuite une requête HTTP : (POST /auth/connecter) contenant les informations d'authentification ainsi qu'un token reCAPTCHA.

Le backend (Spring Boot) commence par vérifier la validité du reCAPTCHA en communiquant avec le service externe Google.

- Si le reCAPTCHA est invalide, une erreur (401/400) est renvoyée au frontend.
- Si la vérification réussit, le backend procède à l'authentification de l'utilisateur en vérifiant les identifiants dans la base de données (MySQL), avec un mot de passe chiffré via BCrypt.

Une fois l'utilisateur validé, le système :

- Récupère ses rôles et permissions,
- Génère un access token JWT contenant les informations d'autorisation (claims),
- Crée un refresh token stocké en base de données (associé à l'utilisateur, IP et User-Agent).

Le backend renvoie alors :

- Un access token (dans la réponse JSON),
- Un refresh token via un cookie sécurisé HTTP-only.

2.4.2 Utilisation du JWT

Après authentification, toutes les requêtes API utilisent l'en-tête : (Authorization: Bearer access_token), ce mécanisme permet de sécuriser les endpoints sans nécessiter de réauthentification à chaque requête.

2.4.3 Rafraîchissement du token (Refresh Token)

Lorsque l'access token expire, le frontend envoie une requête : (POST /auth/rafraichir-jeton), en utilisant le refresh token stocké dans un cookie.

Le backend :

- Vérifie la validité du refresh token,
- Contrôle certaines contraintes (IP, User-Agent),
- Applique une stratégie de rotation des refresh tokens,
- Génère un nouveau access token,
- Génère un nouveau refresh token.

La réponse contient :

- Un nouvel access token,
- Un nouveau cookie refresh token.

2.4.4 Déconnexion (Logout)

Lorsque l'utilisateur se déconnecte, une requête est envoyée : (POST /auth/deconnexion)

Le backend :

- Invalide (révoque) la famille des refresh tokens en base,
- Supprime le cookie côté client.

Cela garantit qu'aucune session ne peut être réutilisée après la déconnexion.

2.4.5 Aspects de sécurité

Ce mécanisme assure un haut niveau de sécurité grâce à :

- L'utilisation de JWT signés pour l'authentification,
- La séparation entre access token (court terme) et refresh token (long terme),
- La protection contre les bots via reCAPTCHA,
- Le stockage sécurisé des mots de passe avec BCrypt,
- La rotation des refresh tokens pour limiter les risques de compromission.

2.5 Diagramme de composants

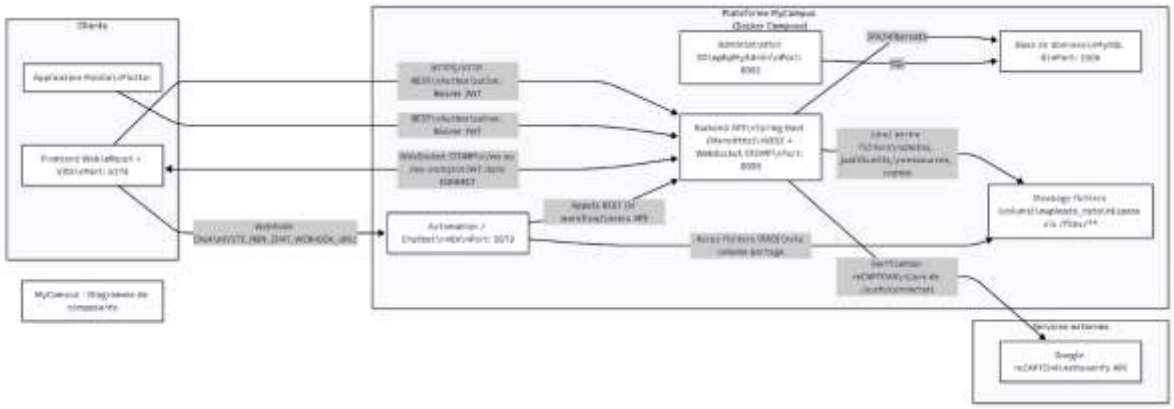


Figure 14 : Diagramme de composants

Le diagramme de composants de **MyCampus** représente l'architecture logicielle globale du système en mettant en évidence les différents modules, leurs responsabilités ainsi que les interactions entre eux.

Cette architecture suit un modèle **client–serveur** basé sur une séparation claire entre :

- la couche présentation (frontend web et mobile),
- la couche métier (backend),
- la couche de données (base de données et stockage fichiers),
- et les services externes (n8n, reCAPTCHA).

2.5.1 Composants côté client

- Frontend Web (React + Vite)

Ce composant constitue l'interface principale accessible via navigateur.
Il permet aux utilisateurs (administrateur, professeur, étudiant) d'interagir avec le système à travers :

- tableaux de bord,
- gestion des données académiques,
- consultation des informations,
- interaction avec le chatbot.

Le frontend communique avec le backend via :

- des requêtes HTTP REST (API),

- des connexions WebSocket pour les notifications en temps réel.

- Application Mobile (Flutter)

L'application mobile offre une expérience multiplateforme (Android/iOS). Elle consomme les mêmes API REST que le frontend web, garantissant ainsi :

- la cohérence des fonctionnalités,
- la réutilisation de la logique métier,
- une synchronisation des données en temps réel.

2.5.2 Composant Backend (Spring Boot)

Le backend constitue le **cœur du système**. Il centralise toute la logique métier et assure plusieurs responsabilités :

- API REST

Expose des endpoints sécurisés pour :

- l'authentification,
- la gestion des utilisateurs,
- la gestion académique,
- la communication (annonces, notifications).

- Gestion de la sécurité
  - Authentification via **JWT**
  - Autorisation via **RBAC**
  - Protection des endpoints avec **Spring Security**
- WebSocket

Permet la communication temps réel pour :

- notifications,
- mises à jour dynamiques.

- Gestion des fichiers
  - Upload (justificatifs, ressources pédagogiques)
  - Download sécurisé via /files/**

2.5.3 Composant Base de données (MySQL)

Ce composant assure la persistance des données :

- Données utilisateurs (authentification, rôles)
- Données académiques (classes, modules, notes)
- Données pédagogiques (présences, examens)
- Tokens (refresh tokens)

L'accès à la base est réalisé via :

- Spring Data JPA / Hibernate (ORM)

2.5.4 Stockage de fichiers

Un système de fichiers (volume Docker) est utilisé pour stocker :

- les documents pédagogiques,
- les justificatifs d'absence,
- les copies corrigées,
- les images (profil).

Ce stockage est séparé de la base de données pour :

- améliorer les performances,
- simplifier la gestion des fichiers volumineux.

2.5.5 Composant n8n (Chatbot RAG)

Le service n8n joue un rôle clé dans l'intelligence du système.

Fonctions principales :

- Réception des requêtes utilisateur via webhook
- Traitement des questions
- Récupération de données depuis :
  - la base MyCampus
  - les documents PDF
- Génération de réponses via approche **RAG**

Communication :

- Frontend → n8n via webhook HTTP
- n8n → Backend via API REST
- n8n → fichiers via volume partagé

2.5.6 Service externe : reCAPTCHA

Lors de l'authentification :

- le backend communique avec l'API Google reCAPTCHA
- pour vérifier que l'utilisateur est humain

2.5.7 Flux global de communication

- L'utilisateur interagit avec le frontend (web/mobile)
- Le frontend envoie des requêtes au backend
- Le backend traite la logique métier
- Les données sont récupérées depuis MySQL ou le stockage fichiers
- Les réponses sont renvoyées au frontend
- Les notifications temps réel passent par WebSocket
- Les requêtes intelligentes passent par n8n

2.6 Diagramme de déploiement

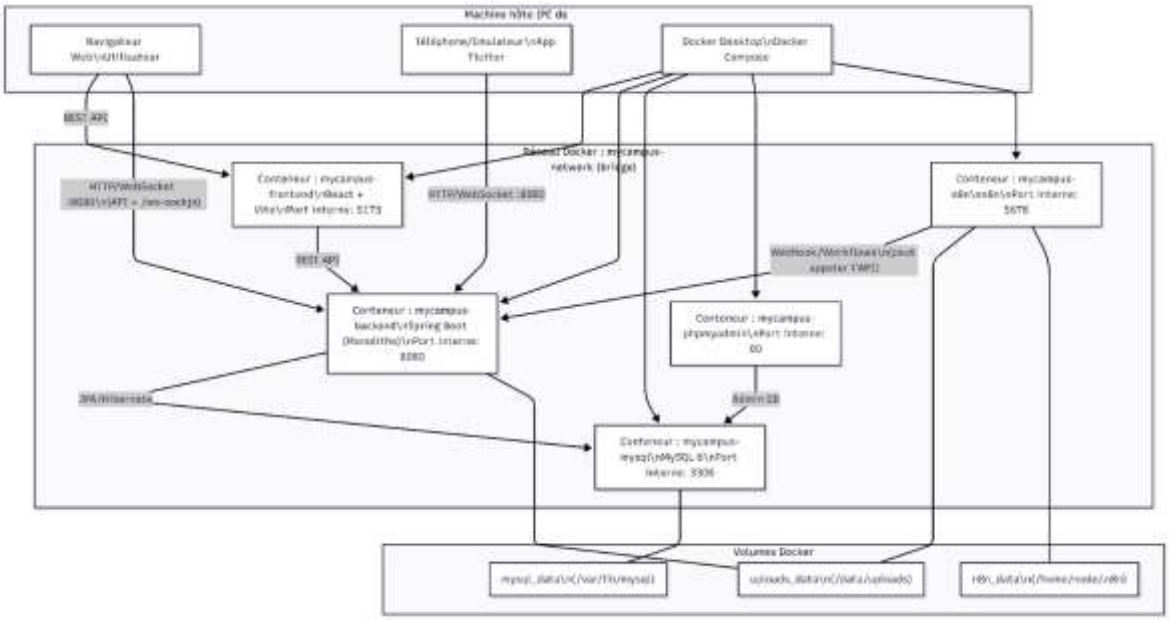


Figure 15 : Diagramme de déploiement

Le diagramme de déploiement présente l'architecture physique de MyCampus et la manière dont les différents composants sont déployés dans un environnement d'exécution basé sur Docker Compose.

Il montre la distribution des services, leurs interactions réseau ainsi que les mécanismes de persistance des données.

2.6.1 Environnement Docker

L'ensemble du système est déployé sous forme de conteneurs Docker, permettant :

- l'isolation des services,
- la reproductibilité de l'environnement,
- la facilité de déploiement.

Chaque composant est encapsulé dans un conteneur indépendant.

2.6.2 Les conteneurs principaux

- **Conteneur Backend (Spring Boot)**
  - Expose l'API REST (port 8080)
  - Gère la logique métier
  - Communique avec MySQL
  - Gère WebSocket
- **Conteneur Frontend (React)**
  - Expose l'interface utilisateur (port 5173)
  - Communique avec le backend via HTTP/WebSocket
- **Application Mobile**
  - Fonctionne côté client (hors Docker)
  - Accède au backend via Internet ou réseau local
- **Conteneur Base de données (MySQL)**
  - Stocke toutes les données persistantes
  - Accessible uniquement par le backend
  - Protégé dans le réseau Docker
- **Conteneur phpMyAdmin**
  - Interface d'administration de la base
  - Permet la gestion manuelle des données
- **Conteneur n8n**

- Gère les workflows automatisés
- Expose un webhook pour le chatbot
- Interagit avec backend et fichiers

2.6.3 Gestion des volumes (persistance)

Le système utilise des volumes Docker pour assurer la persistance :

- **mysql_data** → données de la base
- **uploads_data** → fichiers utilisateurs
- **n8n_data** → workflows et configurations

Cela garantit que les données ne sont pas perdues même si les conteneurs sont recréés.

2.6.4 Réseau Docker

Tous les conteneurs communiquent via un réseau interne Docker :

- Backend ↔ MySQL
- Backend ↔ n8n
- Frontend ↔ Backend

Cela assure :

- une communication sécurisée,
- une isolation vis-à-vis de l'extérieur.

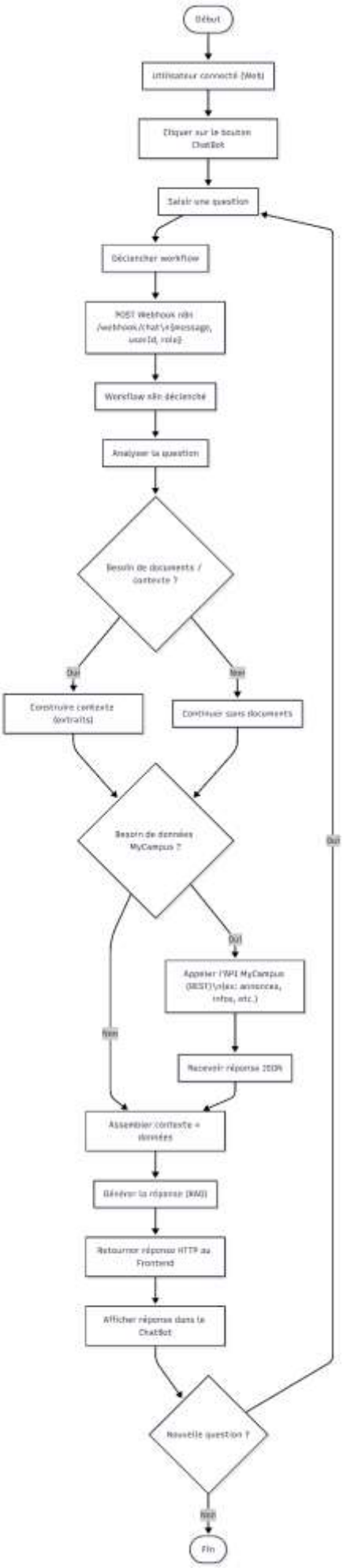
2.6.5 Flux de déploiement

- L'utilisateur lance : docker-compose up --build
- Docker démarre :
  - la base de données
  - le backend
  - le frontend
  - les services annexes
- Les services deviennent accessibles :
  - Frontend → navigateur
  - Backend → API
  - phpMyAdmin → admin BD
  - n8n → chatbot

2.6.6 Sécurité du déploiement

- Isolation des services via conteneurs
- Accès restreint à la base de données
- Utilisation prévue de HTTPS en production
- Gestion des variables d'environnement (secrets)

2.7 Diagramme d'activité



Ce diagramme d'activité décrit le fonctionnement du chatbot MyCampus via n8n. Après connexion, l'utilisateur ouvre le chatbot et saisit une question, puis le frontend déclenche le workflow en envoyant une requête au webhook n8n (/webhook/chat) avec le message, l'identifiant utilisateur et le rôle. Le workflow analyse ensuite la question et décide si un contexte documentaire est nécessaire : si oui, il récupère et construit des extraits à partir des documents disponibles, sinon il continue sans documents. Le processus vérifie ensuite si des données MyCampus sont requises ; dans ce cas, n8n appelle l'API MyCampus et récupère la réponse JSON. Enfin, n8n assemble le contexte (documents et/ou données), génère la réponse (RAG), renvoie la réponse au frontend qui l'affiche à l'utilisateur, puis le cycle peut recommencer tant que l'utilisateur pose de nouvelles questions.

Figure 16 : Diagramme d'activité

.

Chapitre 3 :

Implémentation :

1 Environnement de développement

1.1 Outils et logiciels.

Outil	Version	Rôle
IntelliJ IDEA / VS Code	2024.x / latest	EDI backend et frontend
Android Studio	2024.x	Développement Flutter mobile
Git + GitHub	2.x	Versionnement et collaboration
Docker Desktop	4.x	Conteneurisation locale
Postman	11.x	Tests des API REST
PhpMyAdmin	8.x / 5.x	Administration base de données
draw.io / StarUML	-	Modélisation UML
N8N Community	2.1.5	Workflows automation + Agent ChatBot RAG

Tableau 5 : Outils de développements

Cette chaîne assure un développement full-stack cohérent : IntelliJ/VS Code pour le code, GitHub pour la collaboration, Docker pour l'environnement reproductible, Postman pour valider les contrats API, et N8N pour l'intelligence conversationnelle. Draw.io complète la documentation UML tandis que phpMyAdmin/MySQL Workbench garantisse la fiabilité des données.



Figure 17 : Logo des outils et logiciels.

2 Technologies utilisées

2.1 Backend

- Java 17
- Spring Boot 4.0.1
- Spring Web (REST), Spring Data JPA (Hibernate), Validation
- Sécurité : Spring Security + OAuth2 Resource Server (JWT)

- JWT (HMAC HS512) + Refresh token (cookie HTTP-only)
- WebSocket : Spring WebSocket + STOMP
- Documentation API : springdoc-openapi 3.0.1 (Swagger/OpenAPI)
- Import fichiers : Apache POI 5.5.1 (XLS/XLSX) + commons-csv 1.10.0
- ReCAPTCHA : validation serveur via RestTemplate (appel API Google)

2.2 Frontend Web

- React 19.2.0, Vite 7.2.4
- Routing : react-router-dom 7.12.0
- UI/Charts : lucide-react, recharts
- Sécurité côté login : react-google-recaptcha
- Style : Tailwind CSS 4.1.18
- Chatbot : composant ChatBot connecté au webhook n8n

2.3 Mobile

- Flutter (Dart SDK ^3.7.2)
- State management : provider
- Stockage sécurisé : flutter_secure_storage
- Cookies : cookie_jar + dio_cookie_manager
- Fichiers : file_picker, open_filex, path_provider, permission_handler, image_picker
- Temps réel : stomp_dart_client (WebSocket STOMP)

3 Description des modules développés

3.1 Authentification et sécurité

- Connexion POST /auth/connecter avec reCAPTCHA
- Rafraîchissement POST /auth/rafraichir-jeton (rotation refresh token)
- Déconnexion POST /auth/deconnexion
- RBAC : rôles (ADMIN, PROFESSEUR, ETUDIANT) + permissions, contrôlées via @PreAuthorize
- Administration RBAC : /api/admin/securite (CRUD rôles/permissions + affectations)

3.2 Gestion de la structure académique (Admin)

- Départements, filières, classes, modules, matières (API REST côté admin)
- Exemple (départements) : /api/admin/structure/departements + actions Ouvrir/Fermer
- Affectation d'enseignement : entité AffectationEnseignement (Professeur–Classe–Matière)

3.3 Gestion des étudiants (Admin)

- Import en masse : POST /api/admin/etudiants/import-fichier (CSV/XLS/XLSX) avec mode PARTIEL ou ATOMIC

3.4 Pédagogie

- Étudiant : consultation modules/matières/séances/ressources, présences, relevé de notes par semestre
- Justification d'absence (upload) : PUT /justification/{presenceId}
- Professeur : gestion des présences par séance (GET/PUT /api/professeur/seances/{seanceId}/presences)
- Professeur : gestion des notes (feuille, autosave, publication) + dépôt/téléchargement de copies corrigées (/api/professeur/notes/...)

3.5 Communication

- Annonces :
  - ✓ Admin : /api/admin/annonces (création, publication/dépublication, suppression, listing)
  - ✓ Professeur : /api/professeur/annonces (annonces pour classes assignées)
  - ✓ Étudiant : /api/etudiant/annonces (annonces visibles : globales + classe)
- Notifications : /api/notifications (liste paginée, compteur non lues, marquer lue, lire tout)
- WebSocket STOMP : endpoints /ws et /ws-sockjs avec authentification JWT sur CONNECT

3.6 Emploi du temps

- Implémentation actuelle basée sur des fichiers JSON dans resources/emplois/classes/

- Étudiant : GET /api/etudiant/emploi/{semester} + GET /api/etudiant/emploi/prochaine
- Admin : GET /api/etudiant/emploi/{codeClasse}/{semester} (permission ADMIN_EMPLOI_GERER)
- Professeur : chargement par filtrage des slots selon professeurPpr (côté métier)

4 Présentation de l'intégration n8n

MyCampus intègre un chatbot dans le frontend web (React). Le chatbot communique avec n8n via un webhook HTTP (endpoint POST /webhook/chat), en envoyant un payload standard : message, userId, role. Le service n8n est déployé avec le projet via Docker Compose (port hôte 5679).



Figure 18 : Workflow de n8n

4.1 Objectif du workflow

Le workflow n8n a pour objectif de fournir une assistance contextualisée :

- **Réponses basées sur les données MyCampus** (profil, notes, présences, séances à venir, annonces, etc.).
- **Réponses basées sur des documents PDF** (cours/ressources), en extrayant le contenu du PDF puis en répondant uniquement à partir de ce contenu (approche RAG).

4.2 Comment ce RAG fonctionne ?

4.2.1 Déclenchement et entrée du workflow

- **Webhook d'entrée**

Le workflow démarre dès réception d'une requête POST sur (http://localhost:5679/webhook/chat)

- **Données reçues**

Le webhook reçoit un objet contenant :

  - message : question de l'utilisateur
  - userId : identifiant de l'utilisateur connecté
  - role : rôle applicatif (ETUDIANT / PROFESSEUR / ADMIN)

4.2.2 Guardrail IN (filtrage avant LLM)

Le nœud Guardrail IN applique un contrôle avant toute exécution :

- Validation de l'entrée : message non vide, longueur limitée (≤ 600), présence de userId et role.
- Détection de patterns non autorisés : tentative d'injection (ex. "ignore previous", "jailbreak", "system:"), marqueurs SQL évidents (ex. DROP TABLE, --).
- Classification de la question : calcul de isDocQuestion à partir de mots-clés (pdf, document, cours, résumé, TD/TP…).

Si le message est jugé non conforme, le workflow retourne immédiatement une réponse d'erreur (sans appeler le modèle).

4.2.3 Logique de routage (If / If-Document)

- **Branche "Message bloqué"**
  - Condition : blocked = true
  - Action : Respond - Erreur (réponse ok=false)
- **Branche "Question document"**
  - Condition : isDocQuestion = true
  - Objectif : rechercher un PDF accessible, extraire le texte et répondre sur le document.

- **Branche “Question non-document”**
  - Condition : isDocQuestion = false
  - Objectif : répondre à partir des données MyCampus extraites de la base.

4.2.4 Branche “Document” (PDF / Cours)

- **Récupération des documents accessibles (MySQL)**

Le workflow exécute une requête SQL sur la base MyCampus afin de récupérer une liste de fichiers (fichier_seance) en respectant l’accès selon le rôle :

  - Étudiant : fichiers liés aux séances de sa classe.
  - Professeur : fichiers liés aux séances dont il est responsable.
  - Admin : accès large aux documents.
- **Sélection automatique du fichier (Choisir Fichier)**

Le workflow choisit un document à analyser :

  - par matière mentionnée dans la question,
  - ou par type (TD / TP / COURS),
  - sinon le fichier le plus récent.
- **Lecture et extraction du PDF**

Le workflow :

  - lit le fichier depuis le disque (volume partagé uploads_data),
  - extrait le texte du PDF,
  - tronque le texte si nécessaire pour rester dans des limites raisonnables.
- **Construction du prompt “document”**

Le prompt impose deux règles :

  - répondre uniquement avec le contenu du PDF,
  - si l’information n’est pas dans le document, le dire clairement.
  - Si l’utilisateur demande un résumé, le prompt produit un résumé structuré (introduction, points clés, à retenir).

4.2.5 Branche “Non-document” (données MyCampus)

- **Construction du contexte utilisateur (MySQL - Contexte Utilisateur)**

Le workflow récupère les données utiles pour répondre, notamment :

- Étudiant : notes (limitées), présences (synthèse), prochaines séances, annonces visibles.
- Professeur : matières affectées + annonces destinées aux professeurs.
- Admin : contexte moins limité (réponse plus générale, logique “accès large”).

- **Construction du prompt “par rôle” (Construire Prompt)**

Le nœud “Construire Prompt” définit des règles strictes :

- répondre uniquement à partir du contexte récupéré,
- ne rien inventer,
- retourner “information non disponible” si absente.

4.2.6 Règles d’accès et confidentialité (par rôle)

- **Étudiant**
  - Accès aux données personnelles académiques : notes, présences, séances à venir, annonces de sa classe/global étudiant.
  - Accès aux PDF/cours disponibles pour sa classe.
  - Interdiction : accès à des informations concernant d’autres étudiants.
- **Professeur**
  - Accès aux matières affectées et aux informations générales utiles (annonces professeurs).
  - Interdiction : exposition des données personnelles des étudiants (notes, emails, téléphones).
- **Administrateur**
  - Accès le plus large sur la plateforme (structure, utilisateurs, annonces, etc.).
  - Interdiction : accès aux mots de passe (de toute façon hashés) et toute tentative d’injection.
- **Règle commune (Guardrail IN)**
  - Toute question hors sujet, tentative d’injection de prompt ou demande d’accès à des secrets d’authentification est bloquée avant l’appel au LLM.

4.2.7 Guardrail OUT (filtrage après LLM)

Après génération de la réponse, le workflow applique un Guardrail OUT qui :

- **évite de renvoyer des informations sensibles (ex. mention de mot de passe ou hash),**
- **garantit une réponse minimale exploitable,**
- **renvoie ensuite la réponse finale au frontend (Respond - OK, ok=true).**

5 Captures d'écran commentées

5.1 Gestion des annonces (Professeur)

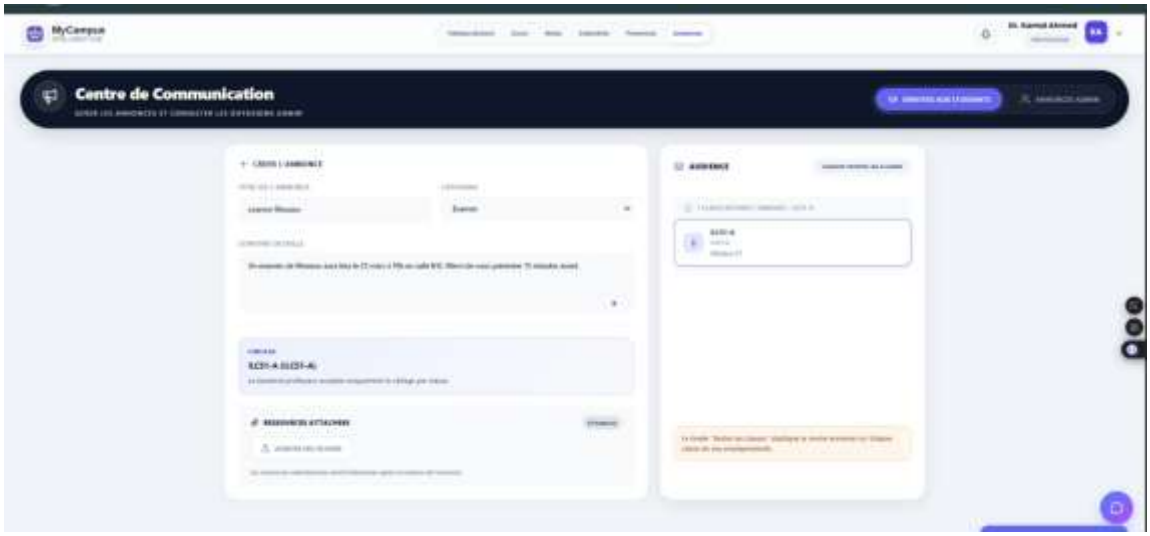


Figure 19 : Page de création d'annonces pour Professeur

création et publication/dépublication d'annonces, visibilité selon rôle (admin global/classe, prof classes assignées).

5.2 Espace Étudiant : présences et justification

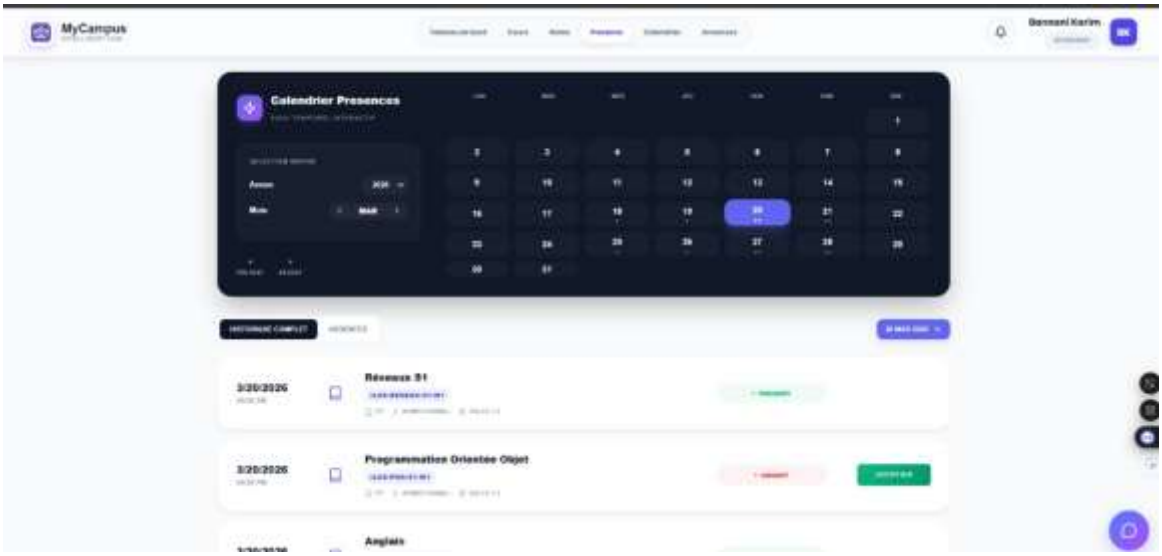


Figure 20 : Page de consultation da la présence d'étudiant

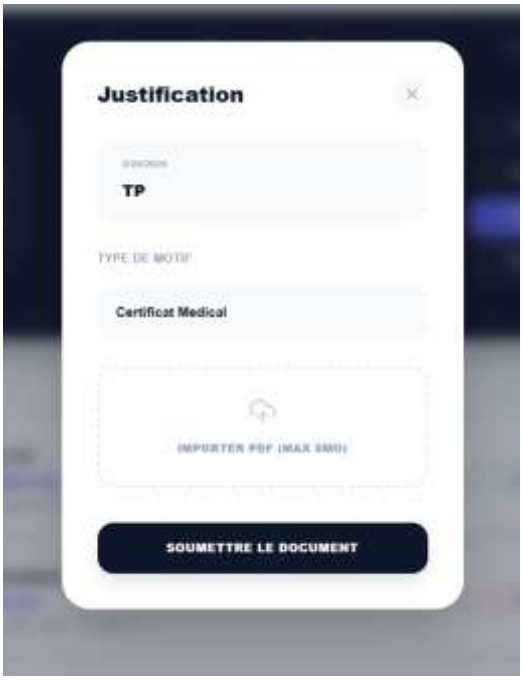


Figure 21 : Page de jusfication d'absence

suivi des absences + dépôt justificatif (fichier) via PUT /justification/{presenceId}.

5.3 ChatBot (web)



Figure 22 : Page du chatbot

composant flottant connecté à n8n via webhook (VITE_N8N_CHAT_WEBHOOK_URL).

.

Chapitre 4 : Sécurité :

1 Analyse des risques

Le système MyCampus traite des informations à caractère sensible, notamment des données personnelles (identité, email, CIN, photo de profil), des données académiques (notes, résultats, absences, relevés) ainsi que des documents numériques (justificatifs d'absence, ressources pédagogiques, copies corrigées). Conformément à une démarche inspirée d'EBIOS Risk Manager et des principes de l'ISO 27005, l'analyse des risques s'appuie sur l'identification des actifs, des menaces, des vulnérabilités et des impacts potentiels sur les trois critères CIA (Confidentialité, Intégrité, Disponibilité). Les scénarios de risque les plus critiques concernent : l'accès non autorisé aux comptes et aux données (usurpation, élévation de privilèges), l'altération des informations (modification frauduleuse des notes/présences/annonces), la divulgation de documents (exposition de fichiers), ainsi que l'indisponibilité du service (surcharge, attaques de type DoS). La présence de plusieurs rôles (Administrateur/Professeur/Étudiant) accroît l'exigence de séparation des privilèges et de contrôle d'accès fin, car une compromission d'un compte à privilèges peut produire des impacts élevés.

2 Politique de sécurité

MyCampus applique une politique basée sur :

- Authentification obligatoire pour toutes les routes métier.
- Autorisation fine par permissions (**RBAC**) au niveau des endpoints.
- Non-exposition des mots de passe et stockage sécurisé via hachage.
- Traçabilité via données de session (IP, User-Agent au niveau refresh token).
- Réduction du risque d'automatisation du login via reCAPTCHA. La documentation API (Swagger/OpenAPI) n'est pas publique : son accès est authentifié.

3 Mécanismes implémentés :

3.1 Authentification

Authentification (JWT / OAuth2 Resource Server)

- Authentification par JWT côté backend (Spring Security + OAuth2 Resource Server).
- JWT signé en HS512, avec un claim scope contenant les authorities/permissions.

- Durée de vie de l'access token : 10 minutes.
- Mécanisme Refresh Token.
- Protection anti-bot sur la connexion : *POST* ***/auth/connecter*** valide un token reCAPTCHA via un appel serveur vers l'API Google.

3.2 Autorisation (RBAC)

- Contrôle d'accès via ***PreAuthorize("hasAuthority('...')")*** sur les controllers.
- API d'administration RBAC disponible : ***/api/admin/securite*** (CRUD rôles/permissions + affectations).

3.3 Chiffrement / stockage sécurisé

- Mots de passe : hachage BCrypt (PasswordEncoder).
- JWT : signature HMAC (HS512). Il ne s'agit pas d'un chiffrement AES/RSA des données applicatives, mais d'une signature garantissant l'intégrité du token.
- Les échanges réseau supposent l'usage de HTTPS en productio.

3.4 Sécurisation WebSocket

- WebSocket STOMP activé (***/ws, /ws-sockjs***).
- Authentification STOMP à la connexion via un intercepteur qui exige Authorization: Bearer <JWT> sur le CONNECT et reconstruit le principal + authorities à partir du JWT.

3.5 Protection contre :

- **Injection SQL**

L'accès aux données est réalisé via Spring Data JPA/Hibernate (repositories), ce qui réduit fortement le risque d'injection SQL par rapport à des requêtes SQL concaténées. La protection reste dépendante des validations et de l'absence de requêtes natives non sécurisées.

- **XSS**

Côté web, l'interface est en React, qui échappe généralement le HTML injecté dans le rendu par défaut. Côté backend, les réponses sont majoritairement en JSON. Une vigilance reste nécessaire sur l'affichage de contenus saisis (ex. annonces) si du HTML est injecté côté frontend.

- **CSRF**

La configuration Spring Security désactive CSRF (csrf.disable()). L'API est principalement consommée via Authorization: Bearer JWT, ce qui limite l'intérêt des attaques CSRF sur les endpoints protégés par header. En revanche, le refresh token est en cookie (HTTP-only) : le paramètre SameSite=Lax réduit le risque, mais une politique CSRF plus stricte peut être envisagée en production selon le déploiement.

- **Attaques DoS/ bruteforce**

La présence du reCAPTCHA sur la connexion réduit cependant le risque de brute force automatisé sur /auth/connecter.

- **Exposition de fichiers**

Le backend expose GET /files/** en accès public (permitAll) via un mapping vers le répertoire de stockage. Cela facilite le téléchargement, mais nécessite une gouvernance stricte (noms non devinables, contrôle d'accès côté métier avant de publier l'URL, séparation des fichiers publics/privés) pour éviter une fuite de documents sensibles.

4 Tests de sécurité (OWASP ZAP)

4.1 Audit de sécurité avec OWASP ZAP

Afin de valider la robustesse de notre plateforme MyCampus face aux cybermenaces, nous avons réalisé un scan de vulnérabilités à l'aide de l'outil de référence OWASP ZAP (Zed Attack Proxy). Cet outil nous a permis de simuler des attaques réelles et d'identifier les faiblesses potentielles de notre infrastructure.



Figure 23 : Logo de Zap

4.1.1 Méthodologie du scan

Nous avons procédé à deux types d'analyses :

- **Scan passif :** Observation des messages échangés entre le navigateur et le serveur pour détecter des en-têtes de sécurité manquants.
- **Scan actif :** Simulation d'attaques (Injection SQL, Cross-Site Scripting - XSS) pour tester la résistance de nos API Spring Boot.

4.2 Objectif du test

Vérifier la configuration de sécurité et détecter les vulnérabilités visibles (DAST).

4.3 Périmètre et environnement

- Environnement : local (Docker Compose).
- Cibles scannées : Frontend http://localhost:5173 et Backend http://localhost:8080 .
- Type de scan : passif + actif.

4.4 Résumé des résultats

- Nombre total d'alertes détectées.
- Répartition par niveau : High / Medium / Low / Informational.



Figure 24 : Resultat de scan par Zap

4.5 Détails des alertes détectées

4.5.1 Alertes critiques (High)

- **Résultat** : 0 Alerte.
- **Analyse** : L'absence d'alertes de niveau "High" indique que les vulnérabilités majeures de type Injection SQL ou Remote Code Execution (RCE) sont

absentes. Cela valide la robustesse de la couche service et de l'ORM (JPA/Hibernate) utilisés dans votre backend.

4.5.2 Alertes importantes (Medium)

- **Content Security Policy (CSP) Header Not Set**

  - **Description** : La politique de sécurité du contenu (CSP) est absente des en-têtes de réponse HTTP.
  - **Impact** : Sans CSP, un attaquant peut exploiter une faille XSS pour injecter et exécuter des scripts malveillants dans le navigateur de l'étudiant. Cela permet de voler des jetons de session (LocalStorage/Cookies) ou de rediriger l'utilisateur vers des sites de phishing.
  - **Résolution** : Configurer Spring Security pour envoyer l'en-tête Content-Security-Policy.

    Exemple : default-src 'self'; script-src 'self' https://trustedscripts.com;

- **Missing Anti-clickjacking Header (X-Frame-Options)**

  - **Description** : L'en-tête X-Frame-Options n'est pas configuré.
  - **Impact** : Un attaquant peut charger l'interface MyCampus dans une <iframe> transparente par-dessus un autre site. L'utilisateur pense cliquer sur le site visible, mais il clique en réalité sur des boutons de votre application (ex: "Supprimer mon compte" ou "Valider une absence").
  - **Résolution** : Ajouter l'en-tête X-Frame-Options: DENY ou SAMEORIGIN dans la configuration des en-têtes de Spring Boot.

4.5.3 Alertes faibles (Low)

- **Strict-Transport-Security (HSTS) Header Not Set**

  - **Description :** Le serveur n'indique pas au navigateur qu'il doit communiquer exclusivement via HTTPS.
  - **Impact :** Un attaquant peut réaliser une attaque de type "SSL Striping" en interceptant la connexion et en forçant l'utilisateur à passer du HTTPS vers le

HTTP simple (non chiffré), permettant de lire les mots de passe en clair sur le réseau.

- **Résolution :** Activer l'en-tête Strict-Transport-Security avec une durée de validité (max-age). Note : Cela nécessite l'installation d'un certificat SSL (même auto-signé en local).

- **X-Content-Type-Options Header Missing**

  - **Description :** L'en-tête de protection contre le "MIME Sniffing" est absent.
  - **Impact :** Le navigateur pourrait ignorer le type de fichier envoyé par le serveur et essayer de le deviner. Un attaquant pourrait envoyer un fichier texte contenant du code JavaScript malveillant, et le navigateur l'exécuterait en pensant que c'est un script.
  - **Résolution :** Forcer le navigateur à respecter le type MIME envoyé par le serveur avec l'en-tête X-Content-Type-Options: nosniff.

4.5.4 Alertes informatives (Informational)

- **Information Disclosure (Divulgation d'informations)**

  - **Description** : Présence de commentaires dans le code source ou d'en-têtes révélant les versions des technologies (ex: Server: Apache/2.4.x).
  - **Impact :** Aide un attaquant à identifier les versions précises de vos outils pour chercher des failles spécifiques (CVE) connues sur ces versions.
  - **Résolution :** Désactiver les bannières de version dans les fichiers de configuration du serveur et nettoyer les commentaires de développement dans le build de production de React.

4.6 Synthèse des remédiations

Vulnérabilité	Risque	Correction Technique (Spring Security)
CSP Missing	Vol de session (XSS)	.contentSecurityPolicy(csp -> csp.policyDirectives(...))
Clickjacking	Détournement de clic	.frameOptions(frame -> frame.deny())
MIME Sniffing	Exécution de scripts	.contentTypeOptions(withDefaults())
HSTS	Interception de trafic	.httpStrictTransportSecurity(hsts -> hsts.includeSubDomains(true)...)

Chapitre 5 : : Tests et validation :

1 Plan de test : Types de tests réalisés

1.1 Introduction

Dans tout système d'information à vocation institutionnelle, la qualité logicielle constitue un enjeu central. Cette exigence est encore plus marquée dans le cadre du projet MyCampus, qui vise à fournir une plateforme unifiée de gestion universitaire intégrant l'authentification des utilisateurs, la gestion pédagogique, le suivi des présences, l'organisation des examens, la publication des notes ainsi que la diffusion d'annonces académiques. En raison de la diversité de ces fonctionnalités et de la sensibilité des données manipulées, la validation du backend ne peut pas se limiter à une vérification manuelle des écrans ou à des essais ponctuels. Elle doit reposer sur une stratégie de tests structurée, capable de vérifier à la fois la cohérence métier, la sécurité des accès et la stabilité du système dans le temps.

Afin de garantir la fiabilité de l'application, une approche de tests unitaires a été adoptée au niveau du backend Spring Boot. Cette stratégie a pour objectif principal de valider les traitements critiques de manière isolée, sans dépendre d'une base de données réelle ni d'un environnement externe. Elle permet ainsi de contrôler la robustesse des règles métier, de détecter rapidement les régressions après modification du code et de démontrer, dans une logique académique et professionnelle, que les scénarios sensibles du système ont été anticipés et maîtrisés.

Le présent chapitre expose la stratégie de tests retenue dans MyCampus, les outils utilisés, les modules couverts, les scénarios validés ainsi que les résultats obtenus. Une attention particulière est accordée aux règles d'autorisation métier, notamment dans les modules manipulant les séances, les présences, les examens et les notes, où un professeur ne doit agir que sur ses propres données. Cette logique constitue une validation critique tant sur le plan fonctionnel que sur le plan de la sécurité applicative.

1.1.1 Tests unitaires

Les tests unitaires mis en place dans MyCampus reposent principalement sur JUnit 5 et Mockito. Ce choix s'inscrit dans les pratiques recommandées pour les projets Spring Boot, en particulier lorsqu'il est nécessaire de tester de manière ciblée les services métier et les composants applicatifs sans mobiliser l'ensemble de l'infrastructure technique.

JUnit 5 a été retenu comme framework principal de test en raison de sa modernité, de sa lisibilité et de sa bonne intégration avec l'écosystème Spring. Il permet de structurer les scénarios de validation, d'exprimer clairement les cas nominaux ainsi que les cas d'erreur, et de produire des tests facilement compréhensibles et maintenables. Son usage favorise une écriture rigoureuse des assertions et une organisation claire des suites de tests.

Mockito a été utilisé pour isoler la classe testée de ses dépendances, notamment les repositories, les services techniques et les composants de sécurité. Cette approche permet de simuler le comportement de la couche d'accès aux données et des services annexes afin de vérifier uniquement la logique métier portée par la classe testée. Dans cette configuration, les dépendances sont injectées sous forme de mocks via @Mock, tandis que la classe testée est instanciée par @InjectMocks. Cette méthode permet de valider avec précision les interactions attendues, d'assurer qu'une écriture en base n'a pas lieu lorsqu'un accès est interdit, et de tester explicitement les exceptions métiers levées dans les cas non autorisés.

L'exécution des tests a été assurée à l'aide du plugin Maven Surefire, invoqué via la commande mvn test. Ce mécanisme permet d'automatiser la compilation et l'exécution de la suite de tests lors des phases de validation du projet. Il contribue ainsi à une démarche d'assurance qualité reproductible, intégrable dans un processus d'évolution continue du backend.

- **Tests de l'authentification et de la sécurité**

Le module d'authentification occupe une place centrale dans MyCampus, car il conditionne l'accès à l'ensemble des services selon le profil de l'utilisateur. Dans l'implémentation actuelle, la logique de connexion et de gestion de session est portée par le composant SecurityController.java, testé à travers la classe SecurityControllerTest.java.

Cette campagne de tests a permis de vérifier les mécanismes critiques de connexion, de validation du reCAPTCHA, de limitation des tentatives, de génération du jeton d'accès JWT et de rotation du refresh token. Ces scénarios constituent une validation essentielle, car ils conditionnent à la fois la sécurité des accès et l'expérience d'utilisation de la plateforme.

ID	Classe de test	Cas de test	Résultat attendu
AUTH-01	SecurityControllerTest	Connexion avec identifiants valides et reCAPTCHA valide	Génération d'un access token et création du cookie de refresh token
AUTH-02	SecurityControllerTest	Échec de connexion lorsque le reCAPTCHA est invalide ou absent	Levée d'une exception de sécurité et enregistrement d'un échec
AUTH-03	SecurityControllerTest	Échec de connexion lorsque le mot de passe est incorrect	Rejet de l'authentification et enregistrement d'un échec
AUTH-04	SecurityControllerTest	Rafraîchissement de session avec refresh token valide	Retour d'un nouveau JWT et mise à jour du cookie
AUTH-05	SecurityControllerTest	Tentative de rafraîchissement sans cookie de refresh token	Rejet de la demande avec message d'erreur explicite

Tableau 6 : Tests de l'authentification et de la sécurité

Ces tests permettent de confirmer que le mécanisme d'authentification ne se contente pas d'accepter ou de rejeter des identifiants, mais applique également des contrôles complémentaires destinés à limiter les abus. Cette approche permet de consolider la sécurité dès le niveau unitaire, avant même l'intégration avec le frontend ou les clients mobiles.

1.1.2 Tests des présences

Le module de gestion des présences joue un rôle pédagogique et administratif majeur dans MyCampus. Il permet au professeur de charger une feuille d'appel, de saisir la présence ou l'absence des étudiants, puis de traiter les justificatifs éventuels. Ce module est également sensible du point de vue de l'autorisation, puisqu'un enseignant ne doit accéder qu'aux séances qui lui appartiennent réellement.

Dans le backend actuel, ce comportement est implémenté dans la classe GestionPresencesProfesseurMetierImpl.java, testée à travers GestionPresencesProfesseurMetierImplTest.java. Les tests ont porté sur le chargement de la feuille d'appel, la mise à jour d'une présence existante, le refus d'enregistrement lorsqu'un étudiant n'appartient pas à la classe concernée, ainsi que la validation ou le refus d'un justificatif selon l'état réel des données disponibles.

ID	Classe de test	Cas de test	Résultat attendu
PRES-01	GestionPresencesProfesseurMetierImplTest	Chargement de la feuille d'appel pour une séance autorisée	Retour de la liste des étudiants avec les présences existantes et les absences non encore saisies
PRES-02	GestionPresencesProfesseurMetierImplTest	Enregistrement d'une présence déjà existante avec passage à l'état présent	Mise à jour de la présence et réinitialisation du justificatif
PRES-03	GestionPresencesProfesseurMetierImplTest	Tentative d'enregistrement pour un étudiant hors de la classe de la séance	Levée d'une exception métier et absence de sauvegarde
PRES-04	GestionPresencesProfesseurMetierImplTest	Validation d'un justificatif inexistant ou incomplet	Rejet de l'opération avec exception explicite
PRES-05	GestionPresencesProfesseurMetierImplTest	Refus d'un justificatif existant	Mise à jour du statut justificatif à l'état refusé

Tableau 7 : Tests des présences

Cette série de tests constitue une validation critique, car elle garantit que les présences ne peuvent pas être manipulées arbitrairement. Elle vérifie également qu'une opération de saisie incomplète ou incohérente ne provoque pas d'écriture non légitime dans la couche de persistance.

1.1.3 Tests des examens et des notes

Le module examens et résultats représente l'un des noyaux fonctionnels les plus sensibles de MyCampus. Il couvre la planification des évaluations, la gestion des corrigés, la saisie ou mise à jour des notes, la publication des résultats ainsi que le contrôle strict de l'accès à ces données. Toute erreur dans cette partie du système aurait un impact direct sur la crédibilité académique de la plateforme.

Dans l'architecture actuelle, ce domaine fonctionnel est porté par deux classes distinctes : GestionExamensProfesseurMetierImpl.java et GestionNotesProfesseurMetierImpl.java, testées respectivement à travers GestionExamensProfesseurMetierImplTest.java et GestionNotesProfesseurMetierImplTest.java.

Les tests ont couvert la création d'un examen dans un contexte d'affectation valide, le rejet d'un doublon métier, le refus de modification d'un examen par un professeur non autorisé, la protection contre la suppression d'un examen déjà utilisé, la création d'un résultat inexistant lors de la première saisie de note, la publication des résultats et le refus des modifications lorsque l'accès au contexte pédagogique est interdit.

ID	Classe de test	Cas de test	Résultat attendu
EXAM-01	GestionExamensProfesseurMetierImplTest	Création d'un examen avec affectation valide	Création d'un examen à l'état programmé
EXAM-02	GestionExamensProfesseurMetierImplTest	Création d'un examen déjà existant pour la même matière, semestre, numéro et type	Rejet de la création avec exception métier
EXAM-03	GestionExamensProfesseurMetierImplTest	Modification d'un examen par un professeur non affecté à la matière	Refus d'accès et absence de sauvegarde

EXAM-04	GestionExamensProfesseurMetierImplTest	Suppression d'un examen pour lequel des résultats existent déjà	Rejet de l'opération et absence de suppression
EXAM-05	GestionExamensProfesseurMetierImplTest	Dépôt d'un nouveau corrigé à la place d'un ancien	Sauvegarde du nouveau fichier et suppression silencieuse de l'ancien
NOTE-01	GestionNotesProfesseurMetierImplTest	Chargement de la feuille de notes pour une classe autorisée	Retour des lignes de notes et des statistiques de saisie
NOTE-02	GestionNotesProfesseurMetierImplTest	Saisie d'une note lorsqu'aucun résultat n'existe encore	Création d'un nouveau résultat
NOTE-03	GestionNotesProfesseurMetierImplTest	Saisie d'une note par un professeur non autorisé	Rejet de l'opération et absence de persistance
NOTE-04	GestionNotesProfesseurMetierImplTest	Publication des notes d'un examen	Datation des résultats publiés et passage de l'examen à l'état corrigé

NOTE-05	GestionNotesProfesseurMetierImplTest	Dépôt de copie corrigée pour un étudiant n'appartenant pas à la classe	Refus d'accès et absence de stockage
NOTE-06	GestionNotesProfesseurMetierImplTest	Saisie pour un étudiant inexistant	Rejet de la saisie avec exception explicite

Tableau 8 : Tests des examens et des notes

Au-delà de la validation fonctionnelle, ce module a également été conçu en intégrant une logique de protection contre la faille IDOR (Insecure Direct Object Reference) décrite par l'OWASP. Cette vulnérabilité apparaît lorsqu'un utilisateur peut accéder à une ressource simplement en manipulant son identifiant technique, sans qu'un contrôle d'appartenance ne soit réellement effectué. Dans le contexte de MyCampus, une telle faille pourrait permettre à un professeur de modifier les notes d'un autre enseignant en changeant la valeur de idExamen, ou de consulter une séance étrangère en manipulant idSeance.

La stratégie de tests mise en place vise précisément à prévenir ce risque. Les méthodes critiques ne se contentent pas de rechercher une ressource par son identifiant ; elles vérifient explicitement que cette ressource appartient au contexte pédagogique autorisé pour le professeur connecté. Ainsi, un examen n'est modifiable que si la matière concernée est réellement liée à une affectation active du professeur. De même, une feuille de notes n'est accessible que si la classe et la matière appartiennent bien à son périmètre d'intervention. Cette approche permet de transformer un simple contrôle d'existence en un contrôle d'autorisation métier fort. Ce test constitue une validation critique au regard de la sécurité applicative, car il réduit fortement le risque d'accès horizontal non autorisé entre utilisateurs de même niveau hiérarchique.

1.1.4 Tests des annonces

Le module des annonces permet la publication d'informations académiques à destination des étudiants ou des professeurs selon différents ciblages. Il s'agit d'un composant important du système, car il contribue à la circulation des informations institutionnelles et pédagogiques. Dans le code source actuel, ce domaine est pris en charge par des classes distinctes côté professeur et côté administrateur, notamment GestionAnnoncesProfesseurMetierImpl.java et GestionAnnoncesAdministrateurMetierImpl.java.

Dans le premier lot de tests exécutés, la priorité a été donnée aux modules les plus critiques sur le plan pédagogique et sécuritaire, à savoir l'authentification, les présences, les examens, les notes et les séances. Le module annonces constitue donc une extension logique de la stratégie de test et peut être formalisé, dans la suite du projet, sous forme de classes de test dédiées.

Les scénarios à valider de manière prioritaire pour ce module concernent la création d'une annonce, la récupération des annonces, la suppression d'une annonce inexistante, ainsi que le refus d'accès lorsqu'un utilisateur tente de publier ou de supprimer une annonce hors de son périmètre fonctionnel.

ID	Classe de test visée	Cas de test	Résultat attendu
ANN-01	GestionAnnoncesProfesseurMetierImplTest	Création d'une annonce de classe par un professeur autorisé	Sauvegarde de l'annonce et notification aux destinataires concernés
ANN-02	GestionAnnoncesProfesseurMetierImplTest	Création d'une annonce globale par un professeur	Rejet de l'opération, ce ciblage n'est pas autorisé
ANN-03	GestionAnnoncesProfesseurMetierImplTest	Suppression d'une annonce par un professeur qui n'en est pas l'auteur	Refus d'accès et absence de suppression
ANN-04	GestionAnnoncesAdministrateurMetierImplTest	Suppression d'une annonce inexistante	Levée d'une exception explicite

ANN-05	GestionAnnoncesAdministrateurMetierImplTest	Publication d'une annonce administrative valide	Sauvegarde et diffusion correcte selon la cible choisie

Tableau 9 : Tests des annonces

L'intégration de ces tests dans une seconde itération permettrait d'étendre encore la couverture fonctionnelle du backend tout en conservant la même logique de validation : contrôle métier strict, isolement des dépendances et prévention des accès non autorisés.

1.2 Résultats des tests

L'exécution de la campagne de validation a été réalisée via la commande mvn test, en s'appuyant sur l'intégration Maven Surefire du projet. Cette exécution a permis de lancer automatiquement l'ensemble des tests unitaires disponibles dans le backend et de vérifier leur cohérence avec l'implémentation effective.

Au terme de cette campagne, l'ensemble des tests exécutés s'est terminé avec succès. Le rapport de build a confirmé l'exécution de 28 tests, sans échec, sans erreur et sans test ignoré. Ce résultat valide le bon comportement des modules testés dans les scénarios nominaux, les cas d'erreur, les règles de sécurité et les contrôles d'autorisation métier. Il constitue un indicateur concret de stabilité pour le backend MyCampus.

Au-delà du simple succès d'exécution, ces résultats montrent que les règles sensibles du projet ont été effectivement formalisées sous forme de tests reproductibles. Cette approche permet de sécuriser l'évolution future du système : toute régression introduite dans les services critiques pourra désormais être détectée plus rapidement lors des phases de développement ou de maintenance.

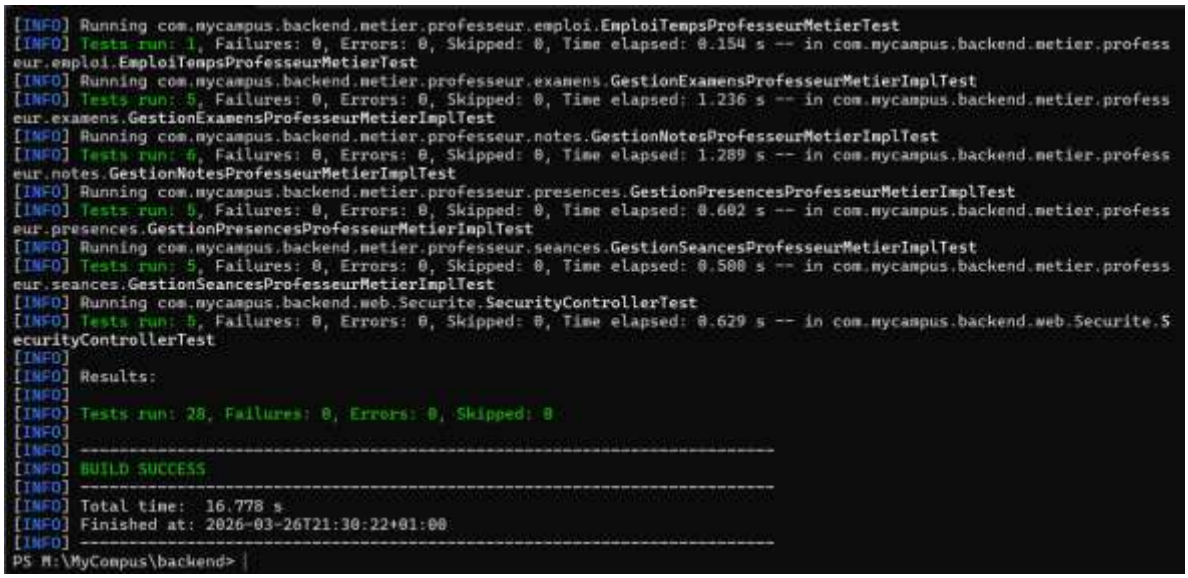


Figure 25 : Résultats des tests d'integration

1.2.1 Conclusion

La mise en place d'une stratégie de tests unitaires dans le backend MyCampus s'inscrit dans une logique d'assurance qualité indispensable à un projet académique sérieux. En s'appuyant sur JUnit 5, Mockito et Maven Surefire, il a été possible de valider de manière structurée les traitements critiques du système, tout en isolant les dépendances techniques afin de se concentrer sur la logique métier réelle.

Cette démarche a permis de vérifier la robustesse de plusieurs modules essentiels, notamment l'authentification, la gestion des présences, les examens, les résultats et les séances. Elle a également permis de confirmer que les contrôles d'accès n'étaient pas uniquement gérés au niveau déclaratif, mais aussi intégrés dans la logique métier elle-même. Cette propriété est particulièrement importante dans un système universitaire, où les droits d'accès doivent refléter fidèlement les responsabilités pédagogiques de chaque acteur.

Enfin, cette stratégie participe pleinement à une conception orientée security by design. En validant explicitement les cas d'accès interdit, l'absence de persistance en cas d'autorisation insuffisante et la protection contre des manipulations d'identifiants de type IDOR, elle renforce non seulement la qualité du code, mais aussi la crédibilité globale du projet devant un jury. Elle constitue ainsi un socle technique solide pour la poursuite du développement et pour l'évolution future de la plateforme MyCampus.

1.3 Les tests d'intégration

Les tests unitaires permettent de vérifier un composant de manière isolée, en contrôlant sa logique interne indépendamment de ses dépendances réelles. Cette démarche est essentielle pour valider les règles métier et prévenir les régressions locales, mais elle ne suffit pas à garantir le comportement global d'une application backend structurée en couches. Dans le cas du projet MyCampus, l'architecture repose sur une séparation entre contrôleur, service métier, repository et base de données. Or, une grande partie de la fiabilité du système dépend précisément de l'interaction correcte entre ces couches.

Il devient donc nécessaire de compléter les tests unitaires par des tests d'intégration, afin de vérifier la circulation réelle des données, la persistance effective en base, le respect des relations JPA et la bonne application des règles de sécurité dans un contexte d'exécution proche de la réalité. Une campagne de tests d'intégration a ainsi été menée sur plusieurs modules critiques du backend MyCampus, notamment les examens, les notes, les présences ainsi que l'authentification et la sécurité.

Sur le plan technique, les tests d'intégration ont été réalisés à l'aide de l'annotation @SpringBootTest, qui permet de charger le contexte Spring Boot complet. Cette configuration assure l'initialisation effective des services métier, des repositories Spring Data JPA, des composants de sécurité ainsi que de la couche Hibernate utilisée pour la persistance. Les tests ont été exécutés avec le profil actif test, en s'appuyant sur une base de données H2 en mémoire, avec l'URL JDBC jdbc:h2:mem:mycampus-test.

Cette approche permet de reproduire le comportement du backend dans un environnement autonome, sans dépendre d'une instance MySQL externe. Elle présente un intérêt majeur dans un cadre académique, car elle autorise des validations réalistes, rapides et reproductibles. Le contexte a également permis l'initialisation de l'EntityManagerFactory, la détection de 23 repositories JPA ainsi que l'exécution automatique des seeders de données pour les rôles, permissions, utilisateurs, professeurs, étudiants, filières, classes, modules, matières, affectations, séances, présences, examens et résultats.

Test N°	Classe d'intégration	Objectif principal
Test N°1	GestionExamensIntegrationTest	Validation du parcours complet de création et gestion d'un examen, avec persistance réelle en base H2
Test N°2	GestionNotesIntegrationTest	Vérification de la saisie, mise à jour et publication de notes, avec cohérence transactionnelle
Test N°3	GestionPresencesIntegrationTest	Validation du chargement de feuille d'appel et enregistrement de présences en base de données
Test N°4	SecurityAuthenticationIntegrationTest	Contrôle du fonctionnement du module d'authentification dans le contexte Spring complet

Figure 26 : Les tests d'intégration

1.3.1 Test N°1 : Validation globale du module des examens

Le premier scénario d'intégration concerne le module des examens, à travers la classe com.mycampus.backend.integration.GestionExamensIntegrationTest. Ce test vise à valider un parcours métier complet lié à la création et à la gestion d'un examen dans un contexte pédagogique réel. L'objectif est de s'assurer qu'une opération initiée au niveau de la couche métier produit bien les effets attendus jusqu'au niveau de la persistance, sans recourir à des composants simulés.

Dans ce scénario, le test vérifie qu'un examen peut être créé dans un contexte valide, en cohérence avec les données pédagogiques déjà présentes dans le système. La validation ne porte pas uniquement sur la réussite apparente de l'opération, mais aussi sur l'écriture effective en base H2. Une fois l'action réalisée, les données sont relues via les repositories afin de confirmer que l'examen a bien été persisté et qu'il reste correctement associé à la matière concernée.

Ce test constitue une validation critique, car il garantit que le module des examens ne se contente pas de produire un résultat logique en mémoire. Il démontre également que les relations JPA/Hibernate sont correctement prises en charge et que les règles métier associées

à la création d'un examen sont bien respectées dans l'ensemble de la chaîne applicative. Dans un système universitaire, où la planification et l'organisation des évaluations doivent rester strictement cohérentes avec la structure pédagogique, cette vérification revêt une importance particulière.



```
@Test
void creerExamen_DoublonMetier_RejecteCreationEtNePersisteRien() {
    long countAvant = examenRepository.count();

    ExamenCreationEntrerDTO dto = ExamenCreationEntrerDTO.builder()
            .semestre(Semestre.S1)
            .dateExamen(Date.from(LocalDateTime.of( year: 2027,  month: 2,  dayOfMonth: 1,  hour: 10,  minute: 0)
                    .atZone(ZoneId.systemDefault()).toInstant()))
            .numeroExamen(1)
            .typeExamen(TypeExamen.NORMAL)
            .duree(90)
            .build();

    assertThrows(
            NotesProfesseurException.class,
            () -> gestionExamensProfesseurMetier.creerExamen(PPR_PROF_1, CODE_CLASSE, MATIERE_PROF_1, dto)
    );

    assertEquals(countAvant, examenRepository.count());
}
```

Figure 27 : Test unitaire N°1

1.3.2 Test N°2 : Validation intégrée du module des notes

Le deuxième scénario d'intégration concerne le module des notes, à travers la classe com.mycampus.backend.integration.GestionNotesIntegrationTest. Ce test permet de vérifier qu'une opération de saisie ou de mise à jour de note est correctement traitée par la couche service, puis réellement persistée dans la base de données à travers la couche repository. Dans le contexte de MyCampus, ce type de validation est essentiel, car la gestion des notes représente une donnée académique sensible, dont l'exactitude et la traçabilité doivent être garanties.

Le scénario retenu consiste à exécuter une opération métier sur les résultats d'un examen, puis à contrôler l'état réel des données en base H2. Cette approche permet de tester simultanément la couche service et la couche repository, sans séparation artificielle. Elle confirme que l'enregistrement d'une note ne reste pas limité à une simple transformation métier, mais qu'il se traduit bien par une persistance effective des résultats.

Au-delà de la simple saisie de note, ce module d'intégration permet également de contrôler l'état global de l'examen lors d'un processus de publication. Cette approche permet de valider

la cohérence transactionnelle entre les résultats enregistrés et le statut de l'examen auquel ils se rapportent. Ainsi, le test ne se limite pas à une vérification fonctionnelle élémentaire ; il constitue une preuve que le backend traite correctement une opération métier complète dans un environnement d'exécution réel.

1.3.3 Test N°3 : Validation intégrée du module des présences

Le troisième scénario d'intégration porte sur le module des présences, à travers la classe com.mycampus.backend.integration.GestionPresencesIntegrationTest. Ce test a pour finalité de vérifier qu'une opération de gestion des présences s'exécute correctement depuis la couche métier jusqu'à la base de données, dans un contexte pédagogique cohérent. Dans MyCampus, le suivi des présences met en jeu plusieurs entités interdépendantes, telles que la séance, la classe, l'étudiant et l'état de présence.

Le scénario d'intégration permet, dans un premier temps, de vérifier le chargement d'une feuille d'appel à partir des données réellement présentes dans la base H2. Cette étape confirme que les relations entre les séances, les classes et les étudiants sont correctement résolues par les repositories et restituées par la couche métier. Dans un second temps, le test valide l'enregistrement ou la mise à jour d'une présence, puis s'assure que cette modification a été effectivement persistée et peut être relue depuis la base.

Ce test constitue une validation critique de la cohérence des données pédagogiques. Il garantit que les informations manipulées par le système reflètent un état réel et vérifiable de la base de données, et non une simple abstraction logique. Dans le contexte d'une plateforme universitaire, cette exigence est particulièrement importante, car la gestion des présences doit rester fiable, traçable et alignée sur l'organisation effective des séances et des groupes d'étudiants.

1.3.4 Test N°4 : Validation du module d'authentification et de sécurité

Le quatrième scénario d'intégration concerne le module d'authentification et de sécurité, à travers la classe com.mycampus.backend.integration.SecurityAuthenticationIntegrationTest. Dans MyCampus, ce module joue un rôle fondamental, puisqu'il conditionne l'accès à l'ensemble des fonctionnalités du système selon le profil de l'utilisateur. Il était donc nécessaire de vérifier son comportement global dans le contexte Spring complet, au-delà d'une simple validation unitaire.

Ce test permet de confirmer que le module d'authentification fonctionne correctement dans le profil test, avec initialisation effective des composants Spring Boot nécessaires à la sécurité de l'application. Il vérifie que le backend traite de manière cohérente les demandes d'authentification dans un environnement où les services métier, les données semées et les composants de persistance sont réellement disponibles.

La valeur de ce test réside également dans la vérification implicite des règles de sécurité qui structurent le backend. En chargeant le contexte Spring complet, il devient possible de s'assurer que le comportement du module d'authentification reste cohérent avec la configuration applicative, le profil actif et les données d'initialisation. Ce scénario constitue ainsi une validation importante de la stabilité du socle de sécurité de MyCampus dans un environnement de test maîtrisé.

1.3.5 Conclusion

Les tests d'intégration constituent un complément indispensable aux tests unitaires dans la stratégie de validation du projet MyCampus. Alors que les tests unitaires permettent de contrôler la logique interne des composants de manière isolée, les tests d'intégration permettent de vérifier leur coopération effective dans le contexte Spring complet, avec persistance réelle des données, interaction entre couches et respect des règles de sécurité. Cette complémentarité est essentielle dans une architecture backend structurée autour des couches contrôleur, service, repository et base de données.

Dans le cadre de MyCampus, cette démarche a permis de renforcer la fiabilité du système en validant des scénarios critiques sur les examens, les notes, les présences et l'authentification. Elle a également permis de contrôler la cohérence transactionnelle des traitements et la stabilité du backend dans un environnement proche de la réalité applicative.

En conclusion, la mise en place des tests d'intégration apporte une garantie supplémentaire de robustesse, de sécurité et de crédibilité technique, ce qui constitue un atout majeur dans le cadre d'un projet de fin d'études. Combinée à la campagne de tests unitaires déjà réalisée, cette stratégie globale de validation démontre une maîtrise rigoureuse de la qualité logicielle et une approche professionnelle du développement backend.

1.4 Tests de Performance

Cette phase vise à évaluer le comportement du backend du système MyCampus sous une charge contrôlée, afin de garantir une expérience utilisateur fluide et réactive, notamment pour les fonctionnalités critiques liées à la gestion des présences.

Les tests de performance ont été réalisés à l'aide de l'outil Apache JMeter, permettant de simuler des utilisateurs virtuels exécutant des scénarios métier réalistes.

1.4.1 Configuration du Test Plan

Le plan de test a été structuré autour d'un Thread Group simulant plusieurs utilisateurs exécutant un scénario complet correspondant au flux métier du module des présences.

Le scénario testé comprend les étapes suivantes :

- Authentification du professeur (Login)
- Recherche d'une séance
- Chargement de la feuille d'appel
- Saisie d'une présence

Chaque requête est exécutée de manière séquentielle, avec des temps de pause (Think Time) introduits entre les actions afin de reproduire un comportement utilisateur réaliste.

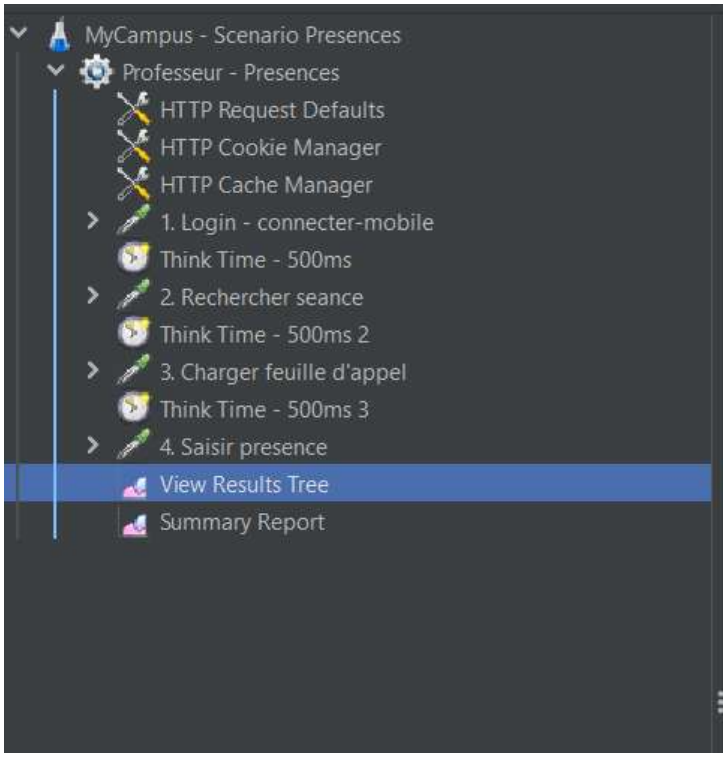


Figure 28 : Configuration du Test Plan

1.4.2 Analyse des métriques de performance

Les tests ont été exécutés sur un total de 80 requêtes (20 itérations par type de requête). Le tableau suivant synthétise les métriques principales observées :

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
1. Login - conne...	20	148	103	808	151.78	0.00%	1.1/sec	1.83	0.27	1782.0
2. Rechercher se...	20	17	9	93	15.50	0.00%	1.1/sec	2.74	1.22	2548.0
3. Charger feuille...	20	19	13	66	11.06	0.00%	1.1/sec	1.34	1.05	1245.2
4. Saisir presence	20	22	17	53	7.58	0.00%	1.1/sec	1.15	1.16	1063.9
TOTAL	80	51	9	808	94.61	0.00%	3.4/sec	5.49	2.85	1659.8

Figure 29 : Analyse des métriques de performance

1.4.3 Résultats et Interprétation

L'analyse des résultats obtenus via JMeter permet d'évaluer la performance globale du backend MyCampus.

- **Fiabilité et Stabilité**

Le taux d'erreur observé est de 0.00% sur l'ensemble des requêtes exécutées, ce qui démontre une excellente stabilité du système.

Toutes les requêtes ont retourné des réponses valides avec un code HTTP 200, confirmant le bon fonctionnement des endpoints testés ainsi que la robustesse du backend face à des appels répétés.



Figure 30 : Fiabilité et Stabilité

- **Interprétation par module**

Module Authentification (Login)

Le temps moyen de réponse de 148 ms est légèrement supérieur aux autres modules, ce qui est attendu. En effet, cette opération implique la vérification des identifiants, le hachage du mot de passe et la génération d'un token JWT. Ces traitements supplémentaires expliquent un temps de réponse plus élevé tout en restant performant.

Module Recherche de séance

Avec un temps moyen de 17 ms, ce module est extrêmement rapide. Cela montre une bonne optimisation des requêtes de lecture ainsi qu'un accès efficace aux données pédagogiques.

Module Chargement de la feuille d'appel

Le temps moyen de 19 ms confirme que le système est capable de charger efficacement les données liées aux étudiants et aux présences, même avec des relations JPA impliquées.

Module Saisie de présence

Le temps moyen de 22 ms indique que les opérations d'écriture en base de données sont rapides et bien maîtrisées. Cela démontre également la bonne gestion des transactions et de la persistance via Hibernate.

1.4.4 Conclusion des Tests de Performance

Les résultats obtenus permettent de conclure que le backend du système MyCampus présente de très bonnes performances globales.

Les principaux points observés sont :

- une excellente stabilité, avec un taux d'erreur nul
- des temps de réponse faibles pour les opérations critiques
- une bonne cohérence transactionnelle lors des opérations d'écriture
- une réactivité adaptée aux usages réels des utilisateurs

La faible variation des temps de réponse indique également que le système reste stable même lors d'exécutions répétées, ce qui garantit une expérience utilisateur fiable.

Ainsi, le système peut être considéré comme performant et prêt à supporter une charge utilisateur modérée, correspondant aux besoins d'une plateforme universitaire.

Conclusion et perspectives

Le présent projet de fin d'études, intitulé MyCampus, s'inscrit dans une volonté de modernisation et de transformation digitale des processus académiques au sein de l'École Supérieure de Technologie de Nador. Au terme de ce travail, nous avons pu concevoir et déployer une plateforme intégrée qui ne se contente pas de numériser des formulaires, mais qui repense globalement l'interaction entre l'étudiant, l'enseignant et l'administration. La réussite de ce projet repose avant tout sur une méthodologie rigoureuse, débutant par une étude de terrain quantitative qui a permis de placer l'utilisateur final au centre de nos préoccupations techniques. En identifiant précisément les points de friction liés à la gestion des notes et au suivi des absences, nous avons pu justifier chaque choix architectural effectué.

Sur le plan technique, ce projet a constitué un véritable défi d'ingénierie logicielle. L'adoption d'une architecture découplée, utilisant Spring Boot pour le backend et React pour le frontend, nous a permis de garantir une séparation nette des responsabilités, facilitant ainsi la maintenance et l'évolution du système. La mise en œuvre de la conteneurisation via Docker a assuré la portabilité de l'application, tandis que l'intégration d'un chatbot intelligent utilisant l'architecture RAG (Retrieval-Augmented Generation) a apporté une dimension innovante, offrant aux utilisateurs un support instantané basé sur les documents réels de l'établissement. Cette fusion entre développement web moderne et intelligence artificielle générative représente, selon nous, l'avenir des systèmes d'information universitaires.

Enfin, en tant qu'étudiants de la filière Ingénierie Logicielle et Cybersécurité, la dimension sécuritaire a été le fil conducteur de notre réalisation. L'utilisation de protocoles d'authentification JWT, la gestion fine des accès par rôles (RBAC) et surtout l'audit de vulnérabilités systématique réalisé avec OWASP ZAP ont permis de livrer une plateforme résiliente. Ce projet nous a non seulement permis de consolider nos acquis théoriques, mais il nous a également confrontés aux réalités de la gestion de projet en équipe, de la résolution de problèmes complexes et de la nécessité d'une veille technologique constante. MyCampus est aujourd'hui une solution prête à l'emploi, capable d'améliorer significativement le quotidien de la communauté estudiantine de l'ESTN.

Bien que les objectifs initiaux de ce PFE aient été atteints, la plateforme MyCampus est conçue comme un système évolutif capable de s'adapter aux futurs défis technologiques de l'enseignement supérieur. La première perspective de développement réside dans la mobilité. Bien que l'interface actuelle soit "Responsive Design", le développement d'une application mobile native (Android et iOS) permettrait d'exploiter les fonctionnalités matérielles des smartphones. L'implémentation de notifications "Push" instantanées pour alerter les étudiants dès la publication d'une note ou d'une annonce officielle augmenterait considérablement la réactivité du système et l'engagement des utilisateurs.

Une seconde perspective majeure, particulièrement adaptée à la dynamisation de la vie estudiantine de l'EST de Nador, concerne la création d'un module de gestion des clubs et activités parascolaires. Ce module permettrait aux bureaux des clubs de gérer leurs adhérents, de publier leurs événements et de réserver des salles ou du matériel via la plateforme. En intégrant cet aspect social, MyCampus deviendrait le point de ralliement unique des étudiants, favorisant non seulement la réussite académique mais aussi l'épanouissement personnel au sein de l'école. Cette centralisation simplifierait également la communication entre les clubs et l'administration pour la validation des projets.

Sur le plan de l'aide à la réussite, l'exploitation des données récoltées à des fins d'analyse prédictive constitue un axe de recherche prometteur. En intégrant des algorithmes de Machine Learning, la plateforme pourrait devenir un outil d'aide à la décision pour la direction des études. Par exemple, le système pourrait analyser les corrélations entre le taux d'absentéisme et les performances académiques pour identifier, de manière préventive, les étudiants en situation de décrochage scolaire. Cette approche permettrait à l'établissement de passer d'une gestion administrative réactive à une stratégie d'accompagnement pédagogique proactive et personnalisée.

Enfin, l'interopérabilité avec les systèmes d'information nationaux via des API dédiées permettrait d'automatiser le flux de données entre l'échelle locale et nationale, réduisant ainsi les risques d'erreurs humaines lors des saisies manuelles. En conclusion, **MyCampus** n'est pas une finalité en soi, mais le socle d'un écosystème numérique ambitieux. Les bases techniques et sécuritaires posées durant ce projet ouvrent la voie à une multitude d'extensions qui feront de l'EST de Nador un modèle d'établissement connecté et intelligent. Nous sommes convaincus que la pérennité de ce projet réside dans sa capacité à évoluer avec les besoins de ses utilisateurs et les innovations de demain.

Référence bibliographique

- Documentation Draw.io :01/12/2026
- Documentation Flutter : 01/02/2026
- Guide de démarrage React :01/01/2026
- Documentation Dart : 01/02/2026

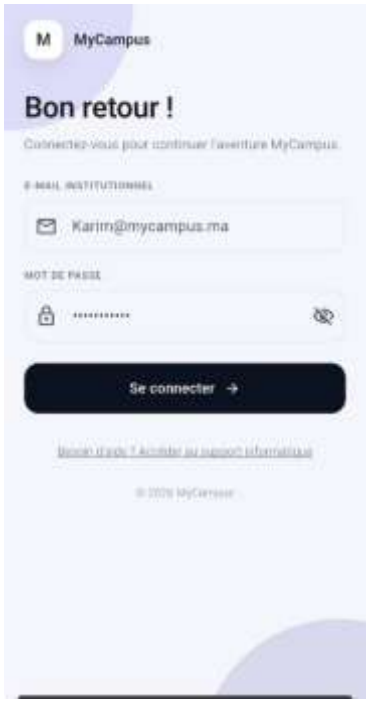
Annexes

Annexe 1 : Page de login de l'application mobile 92
Annexe 2 : L'ensempble des pages de l'étudiant dans la version mobile 93
Annexe 3 : L'ensempble des pages du professeur dans la version mobile 94
Annexe 4 : Page de consultation des annonces pour l'etudiant dans l'application web..... 94
Annexe 5 : Page de gestion des cours pour le professeur dans l'application web 95
Annexe 6 : Diagramme de Gantt 97
Annexe 7 : logo d'outil de gestion de projet 103
Annexe 8 : Estimation des tâches 111

1 Captures d”écran

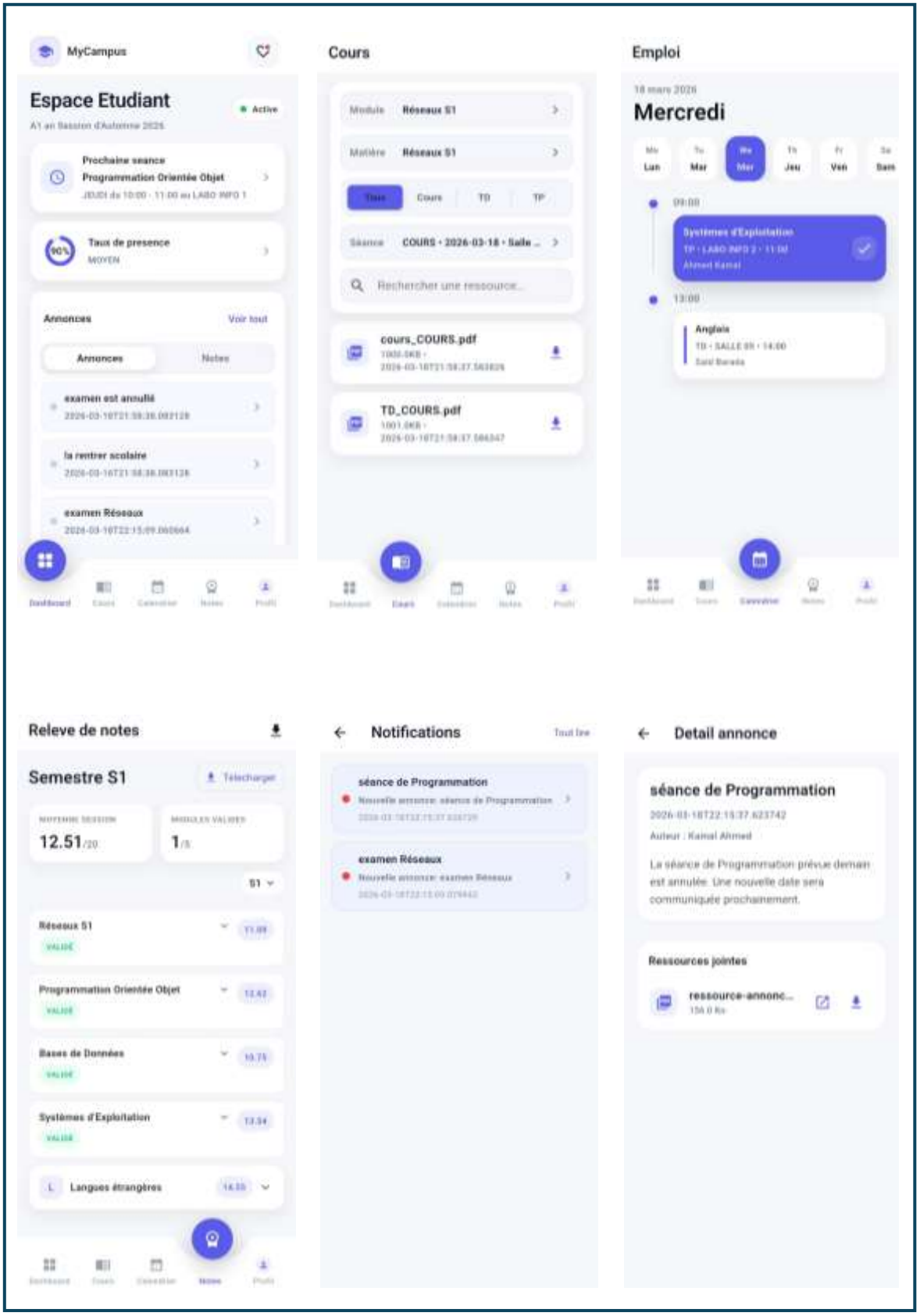
1.1 La version mobile :

Page de connextion dans l’application mobile :



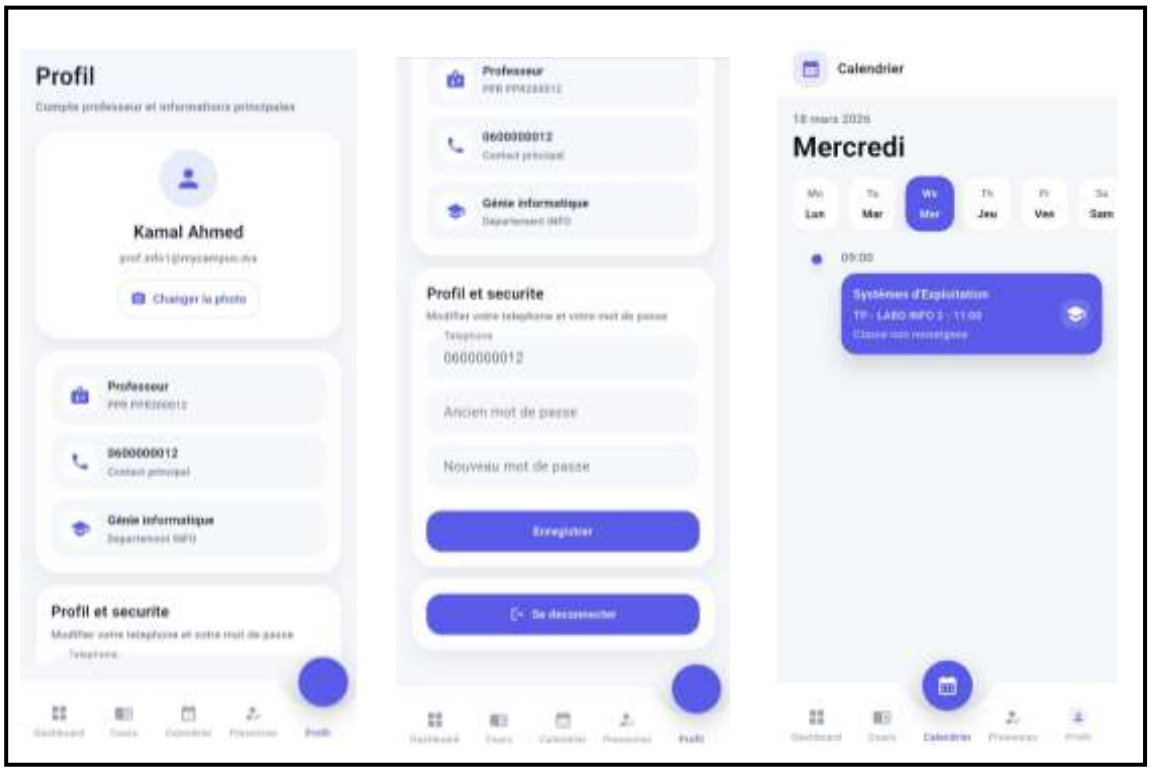
Annexe 1 : Page de login de l'application mobile

- **Etduant :**



Annexe 2 : L'ensempble des pages de l'étudiant dans la version mobile

- **Professeur :**

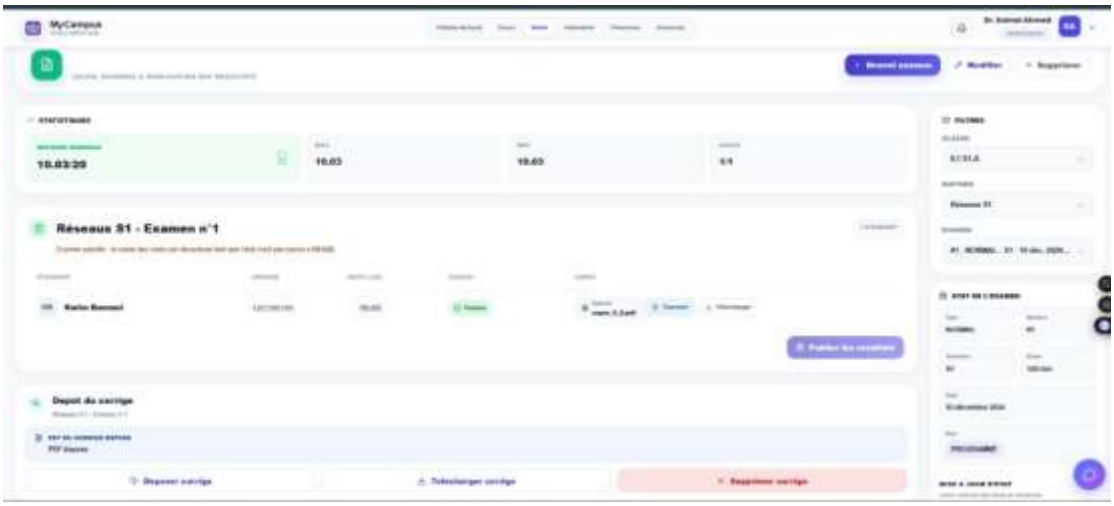


Annexe 3 : L'ensempble des pages du professeur dans la version mobile

1.2 La version web :



Annexe 4 : Page de consultation des annonces pour l'etudiant dans l'application web



Annexe 5 : Page de gestion des cours pour le professeur dans l'application web

2 Code source

2.1 Authentification + Refresh token (cookie HTTP-only)

```
@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class SecurityController {

 @PostMapping("/connecter")
 public JetonsSortieDTO connecter(@Valid @RequestBody ConnectionDTO dto,
                                  HttpServletRequest request,
                                  HttpServletResponse response) {
  if (!recaptchaService.validateRecaptcha(dto.getRecaptchaToken())) {
   throw new SecuriteException("Validation reCAPTCHA échouée.");
  }
  Authentication auth = authenticationManager.authenticate(
   new UsernamePasswordAuthenticationToken(dto.getEmail(), dto.getMotDePasse())
  );
  String jwt = jwtEncoder.encode(jwtEncoderParameters).getTokenValue();
  RefreshToken refresh =
refreshTokenMetier.creerRefreshTokenPourlaPremierFois(auth.getName(), userAgent, ip);
  ResponseCookie cookie = ResponseCookie.from("refresh_token", refresh.getToken())
```

```
.httpOnly(true)
    .sameSite("Lax")
    .path("/auth")
    .maxAge(7 * 24 * 60 * 60)
    .build();
  response.addHeader("Set-Cookie", cookie.toString());
  return JetonsSortieDTO.builder().access_token(jwt).build();
 }
}
```

Explication : l'utilisateur s'authentifie via /auth/connecter avec reCAPTCHA. Le backend génère un JWT court (access token) et un refresh token stocké en cookie HTTP-only pour rafraîchir la session sans réauthentification.

2.2 Workflow n8n : Guardrail IN (anti-injection) + classification "document"

```
let blocked = null;
if (!msg || msg.trim().length === 0) blocked = 'Message vide.';
else if (msg.length > 600) blocked = 'Message trop long.';
else if (!userId || !role) blocked = 'userId et role obligatoires.';
else {
 const forbidden = ['ignore previous','jailbreak','system:','DROP TABLE','--'];
 if (forbidden.some(w => msg.toLowerCase().includes(w.toLowerCase())))
  blocked = 'Message non autorisé.';
}
const docKeywords = ['document','pdf','resume','cours','td','tp','support'];
const isDocQuestion = docKeywords.some(k => msg.toLowerCase().includes(k));
```

Explication : avant d'appeler le LLM, le workflow n8n bloque des entrées suspectes (injection) et détecte si la question doit déclencher la branche "PDF/cours".

3 Diagramme de Gantt :

Ce diagramme de Gantt présente la planification globale du projet **MyCampus** sur la période du **01/11/2025** au **19/03/2026,** avec une organisation par phases et par responsable. La partie gauche du tableau liste les tâches (cadrage et conception, développement backend, développement frontend web, développement mobile/DevOps, puis tests et rapport) ainsi que leurs dates de début/fin, la durée estimée et le membre de l'équipe en charge. La partie droite représente une échelle temporelle hebdomadaire : chaque barre colorée indique la période d'exécution d'une tâche, ce qui permet de visualiser rapidement les chevauchements et le travail en parallèle. Ce planning sert d'annexe pour justifier l'organisation du projet, la répartition des responsabilités et l'enchaînement des livrables.



Task	Start	End	Duration (days)	Owner	Phase
Phase 1 - Cadrage & Conception					
Lancement + organisation + outils	01-nov-2025	01-nov-2025	1	Equipe	Phase 1 - Cadrage & Conception
Recueil besoins + cahier des charges	03-nov-2025	14-nov-2025	12	Equipe	Phase 1 - Cadrage & Conception
Modelisation (UML/ERD) + specifications API/DTO	17-nov-2025	28-nov-2025	12	Equipe	Phase 1 - Cadrage & Conception
Backend (Spring Boot)					
Initialisation backend + config MySQL + couches	01-déc-2025	12-déc-2025	12	Aymane	Backend (Spring Boot)
Auth (JWT HS512) + refresh token + reCAPTCHA	15-déc-2025	10-janv-2026	27	Aymane+Alae	Backend (Spring Boot)
RBAC (roles/permissions) + seeders	12-janv-2026	24-janv-2026	13	Aymane	Backend (Spring Boot)
Structure academique (CRUD)	26-janv-2026	14-févr-2026	20	Alae	Backend (Spring Boot)
Affectations + seances/ressources + fichiers	16-févr-2026	01-mars-2026	14	Alae	Backend (Spring Boot)
Presences + justification (upload)	02-mars-2026	08-mars-2026	7	Alae +Aymane	Backend (Spring Boot)
Notes/examens (feuille, autosave, publier, copies)	09-mars-2026	15-mars-2026	7	Norhane+Alae	Backend (Spring Boot)
Notifications + WebSocket STOMP (JWT CONNECT)	16-mars-2026	19-mars-2026	4	Aymane	Backend (Spring Boot)
Frontend Web (React/Vite)					
Setup web + routing + layouts	01-déc-2025	12-déc-2025	12	Alae	Frontend Web (React/Vite)
Auth UI + session/refresh	15-déc-2025	10-janv-2026	27	Alae	Frontend Web (React/Vite)
Pages Admin (structure + comptes + annonces)	12-janv-2026	14-févr-2026	34	Alae	Frontend Web (React/Vite)
Pages Prof (presences + notes + annonces)	16-févr-2026	08-mars-2026	21	Norhane	Frontend Web (React/Vite)
Pages Etudiant (relève/presences/annonces/emploi)	09-mars-2026	19-mars-2026	11	Alae	Frontend Web (React/Vite)
Mobile + DevOps (Flutter/Docker/n8n)					
Setup Flutter + auth (dio + secure storage + cookies)	01-déc-2025	10-janv-2026	41	Aymane	Mobile + DevOps (Flutter/Docker/n8n
Docker Compose + volumes + doc execution	15-déc-2025	24-janv-2026	41	Aymane+Alae	Mobile + DevOps (Flutter/Docker/n8n
Mobile Etudiant (profil + annonces + notes/relevé)	26-janv-2026	23-févr-2026	29	Aymane	Mobile + DevOps (Flutter/Docker/n8n
Mobile Prof (presences + justification + fichiers)	24-févr-2026	10-mars-2026	15	Norhane	Mobile + DevOps (Flutter/Docker/n8n
STOMP notifications (mobile)	11-mars-2026	19-mars-2026	9	Equipe	Mobile + DevOps (Flutter/Docker/n8n
n8n workflow (webhook + guardrails + RAG PDF)	02-mars-2026	19-mars-2026	18	Alae	Mobile + DevOps (Flutter/Docker/n8n
Phase 2 - Tests & Rapport					
Tests fonctionnels E2E + correction bugs	03-mars-2026	19-mars-2026	17	Equipe	Phase 2 - Tests & Rapport
Redaction rapport + diagrammes + captures	17-févr-2026	19-mars-2026	31	Alae	Phase 2 - Tests & Rapport
Demo interne + preparation soutenance	19-mars-2026	19-mars-2026	1	Equipe	Phase 2 - Tests & Rapport

Annexe 6 : Diagramme de Gantt

4 Manuel utilisateur – MyCampus (Web + Mobile)

4.1 Présentation

MyCampus est une plateforme académique multi-rôles permettant de centraliser la gestion pédagogique et administrative. Elle propose un accès différencié selon le profil Administrateur, Professeur ou Étudiant, avec authentification sécurisée (JWT + refresh token) et contrôle d'accès (RBAC).

4.2 Accès à l'application

4.2.1 Version Web

Ouvrir le navigateur et accéder à : **http://localhost:5173**

4.2.2 API Backend (référence)

API : **http://localhost:8080**

Documentation OpenAPI/Swagger : /swagger-ui (accès authentifié)

4.2.3 Base de données (admin)

PhpMyAdmin : **http://localhost:8081**

4.2.4 n8n (chatbot/workflows)

n8n : **http://localhost:5679**

4.3 Connexion / Déconnexion

4.3.1 Se connecter

- Ouvrir la page de connexion.
- Saisir email et mot de passe.
- Valider le reCAPTCHA.
- Cliquer sur Se connecter.

Résultat : l'utilisateur est redirigé vers l'interface correspondant à son rôle.

4.3.2 Rafraîchir la session

Le rafraîchissement du jeton est géré automatiquement par l'application via le mécanisme refresh token (cookie HTTP-only) lorsque le JWT expire.

4.3.3 Se déconnecter

- Cliquer sur Déconnexion.
- La session est fermée et le cookie de refresh est supprimé.

4.4 Fonctionnalités communes (tous rôles)

4.4.1 Profil utilisateur

- Consulter ses informations

- Modifier certaines informations du profil
- Mettre à jour la photo de profil (upload)

4.4.2 Notifications

- Consulter la liste des notifications
- Voir le compteur des notifications non lues
- Marquer une notification comme lue ou tout marquer comme lu

4.4.3 Chatbot (Web)

Disponible après connexion via un bouton flottant.

Envoie les questions à n8n via webhook, avec le contexte userId et role.

4.5 Guide Étudiant

4.5.1 Tableau de bord Étudiant

Accéder à une synthèse (annonces, accès rapide, etc.).

4.5.2 Espace pédagogique

- Consulter ses modules
- Consulter les matières d'un module
- Consulter les séances d'une matière (filtrage possible par type de séance si proposé dans l'UI)
- Consulter les ressources d'une séance
- Télécharger un fichier de séance (téléchargement sécurisé via API)

4.5.3 Présences / absences

- Consulter la liste des présences/absences (avec filtres possibles selon l'interface).
- Justifier une absence :
  - Sélectionner l'absence concernée.
  - Choisir le type de motif.
  - Ajouter une description (optionnel).
  - Joindre un fichier justificatif (optionnel).
  - Valider l'envoi.

4.5.4 Notes / relevé

Consulter le relevé de notes par semestre (sélection du semestre).

4.5.5 Emploi du temps

- Consulter l'emploi du temps (par semestre selon l'interface).
- Consulter la prochaine séance.

4.5.6 Annonces

Consulter les annonces visibles : annonces globales étudiant + annonces de sa classe.

4.6 Guide Professeur

4.6.1 Tableau de bord Professeur

Accéder à une vue synthèse et à la navigation des modules pédagogiques.

4.6.2 Séances / ressources

Consulter et gérer les contenus liés aux séances (selon les pages disponibles dans l'UI).

4.6.3 Présences

- Charger la feuille d'appel d'une séance.
- Saisir ou modifier les présences des étudiants de la séance.

4.6.4 Notes / examens

- Charger une feuille de notes (classe + examen).
- Saisir/mettre à jour les notes (autosave).
- Publier les notes.
- Déposer une copie corrigée par étudiant et la télécharger si nécessaire.

4.6.5 Annonces Professeur

- Créer une annonce pour les classes qui lui sont affectées.
- Publier/dépublier et supprimer ses annonces.

4.6.6 Emploi du temps

Consulter l'emploi du temps et la prochaine séance (si la page est activée).

4.7 Guide Administrateur

4.7.1 Tableau de bord Administratif

Accéder à une vue globale et aux modules d'administration.

4.7.2 Gestion de la structure académique

- Gérer départements, filières, classes, modules, matières (CRUD).
- Ouvrir/fermer certaines structures (si disponible via l'interface).

4.7.3 Affectations d'enseignement

Gérer les affectations Professeur – Classe – Matière.

4.7.4 Gestion des utilisateurs

Gérer les comptes (étudiants, professeurs, administrateurs) selon les écrans disponibles.

4.7.5 Import des étudiants

- Importer un fichier CSV/XLS/XLSX pour insérer des étudiants en masse.
- Choisir la classe cible et le mode d'import (selon l'interface).

4.7.6 Gestion des annonces (Admin)

- Créer des annonces globales ou par classe.
- Publier/dépublier et supprimer des annonces.

4.7.7 Emploi du temps (Admin)

Consulter l'emploi du temps par classe/semestre (selon l'interface).

4.7.8 Sécurité RBAC

Gérer les rôles, permissions et affectations via le module sécurité (si exposé dans l'interface).

4.8 Bonnes pratiques et sécurité

- Ne pas partager ses identifiants.
- Se déconnecter sur un poste partagé.
- N'uploader que des fichiers nécessaires (justificatifs, ressources) et respecter les formats autorisés par l'application.

4.9 Dépannage (problèmes fréquents)

- Connexion refusée : vérifier reCAPTCHA, identifiants, et que les services Docker sont démarrés.
- Aucune donnée visible : vérifier le rôle et les permissions (RBAC), ou que la base est bien alimentée (seeders/import).
- Téléchargement impossible : vérifier que le fichier existe et que l'utilisateur a le droit d'accès.
- Emploi du temps vide : cela peut arriver si aucun fichier JSON d'emploi du temps n'est disponible pour la classe/semestre.

5 Planification de projet :

5.1 Gestion de Projet avec la Méthode Agile (scrum)

La gestion du projet MyCampus a adopté la méthodologie Agile (Scrum) pour assurer une livraison itérative, flexible et réactive aux besoins académiques changeants. Le projet complexe, impliquant le développement parallèle de trois couches technologiques distinctes (backend Spring Boot, frontend React Web, application mobile Flutter) et couvrant des processus métier complets (structure académique, sécurité RBAC, gestion des examens, présences, notes), a bénéficié d'une approche itérative en sprints de deux semaines. Cette méthodologie a permis à l'équipe de livrer régulièrement des fonctionnalités validées, d'adapter rapidement les priorités face aux découvertes techniques, et de maintenir une qualité logicielle élevée critique pour un système académique manipulant des données sensibles.

5.1.1 Outils Utilisés

Jira a été configuré comme outil central de gestion de projet, servant de single source of truth pour l'état d'avancement du projet. Jira a permis de : créer et organiser les user stories académiques alignées sur les objectifs métier (ex: "En tant qu'étudiant, je peux consulter mes résultats d'examen et télécharger ma relevé de notes"), estimer les efforts via des story points calibrés selon la complexité technique et métier, assigner les tâches granulaires aux développeurs frontend, backend et mobile en fonction de leurs spécialités, suivre l'avancement en temps réel via des sprints avec burn-down charts montrant progression vs prévisions, documenter les blocages (dépendances entre services, problèmes d'intégration BD) et faciliter la synchronisation entre équipes.



Annexe 7 : logo d'outil de gestion de projet

5.1.2 Processus Scrum Implémenté

- ***Planification de Sprint (Sprint Planning)***

Au début de chaque sprint de deux semaines, l'équipe participait à une séance de planification de sprint où : le product owner (ou chef de projet) présentait les stories prioritaires du backlog et le contexte métier, l'équipe technique discutait de la faisabilité de chaque story en termes de dépendances (ex: un feature frontend d'affichage de notes nécessite que l'API backend soit prête), les développeurs estimaient l'effort en story points (échelle 1-13 ou Fibonacci), et les tâches étaient assignées aux contributeurs (backend, frontend, mobile) avec un commitment clair. Pour MyCampus, chaque sprint ciblait des workflows complets : Sprint 1 = structure académique + authentification, Sprint 2 = gestion des examens + API, Sprint 3 = frontend de consultation, etc.

- ***Synchronisations Quotidiennes***

Chaque jour, l'équipe tenait des daily standups de 15 minutes où chaque développeur rapportait :

- Ce qu'il a accompli hier (ex: "API d'ajout de présences testée et déployée"),
- Ce qu'il compte faire aujourd'hui (ex: "intégration frontend de la page de présences"),
- Les blocages rencontrés (ex: "en attente de la spécification du format du JWT du backend").

Ces standups étaient essentiels pour détecter rapidement les dépendances entre équipes (frontend attend l'endpoint backend, mobile attend les data structures finales).

- ***Sprint Reviews et Démonstrations***

À la fin de chaque sprint, un sprint **review** démontrait les features terminées et acceptées par l'équipe : chaque story complétée était démontrée (fonctionnalité visible, tests passant, sécurité validée), les stackholders (encadrants, utilisateurs fictifs) donnaient un feedback (validant que la feature répond au besoin métier), et les items non terminés retournaient au backlog pour le sprint suivant. Pour MyCampus, les démonstrations couvraient : authentification JWT fonctionnant, CRUD complet de structure académique, pages d'affichage des examens/notes/présences, gestion des permissions RBAC.

- ***Rétrospectives (Retours d'Expérience)***

Après chaque sprint, une **rétrospective** permettait à l'équipe d'améliorer son processus : discussions sur "Qu'est-ce qui a bien fonctionné ?", "Qu'est-ce qui aurait pu être mieux ?", "Quels améliorations implémenter au sprint prochain ?". Les améliorations typiques inclus : "Mieux planifier les dépendances API/Frontend", "Automatiser plus de tests unitaires pour détecter bugs tôt", "Clarifier les spécifications métier avant de développer".

5.2 Estimation des taches

Étape	Tâche	Estimation (h/personne)	Risque (30%)	Développeur
Étude préalable 54.6h	Étude de l'existant	6	1.8	Aymane + Norhane + Alae
	Décomposition du projet	12	3.6	
	Estimation des tâches	12	3.6	
	Étude de faisabilité	12	3.6	
Étude Détaillée 125.4h	**Conception (UML)**	48	14.4	Aymane + Norhane + Alae
	Diagramme d'use case	6		
	Diagramme de séquence	6		
	Diagramme de classe	6	8.7	
	Diagramme d'activité	6		
	Diagramme de collaboration	6	6	
	Diagramme d'état-transition	6		
	Diagramme de déploiement	6		
	Étude technique	29		
	Maquette statique	20		
Réalisation 122.5h	❖ **Partie Admin**	**43.5**	**12.9**	Aymane + Alae
	+ Ajouter	**17.5**	9.75	
	Administrateur	1		
	Professeur	1		
	Étudiant	1		
	Département	1		
	Filière	1		
	Classe	1		
	Module	1		
	Matière	1		
	Affectation d'enseignement	1		
	Emploi du temps par classe	2		
	Annonce administrative	1		

	Ressource d'annonce admin	0.5		
	Import des étudiants	3		
	Rôles et permissions	1		
	Test unitaire	0.5		
	+ **Modifier**	**7**		
	Administrateur	0.5		
	Professeur	0.5		
	Étudiant	0.5		
	Département	0.5		
	Filière	0.5		
	Classe	0.5		
	Module	0.5		
	Matière	0.5		
	Affectation d'enseignement	0.5		
	Emploi du temps par classe	0.5		
	Annonce administrative	0.5		
	Ressource d'annonce admin	0.5		
	Rôles et permissions	0.5		
	Test unitaire	0.5		
	+ **Supprimer**	**7**		
	Administrateur	0.5		
	Professeur	0.5		
	Étudiant	0.5		
	Département	0.5		
	Filière	0.5		
	Classe	0.5		
	Module	1		
	Matière	0.5		

	Affectation d'enseignement	0.5		
	Emploi du temps par classe	0.5		
	Annonce administrative	0.5		
	Ressource d'annonce admin	0.5		
	Rôles et permissions	0.5		
	Test unitaire	0.5		
	+ Sélectionner	**12**		
	Tableau de bord administratif	1		
	Liste des administrateurs	0.5		
	Liste des professeurs	0.5		
	Liste des étudiants	0.5		
	Liste des départements	0.5		
	Liste des filières	0.5		
	Liste des classes	2		
	Liste des modules	1		
	Liste des matières	1		
	Liste des affectations d'enseignement	1		
	Emploi du temps par classe	0.5		
	Télécharger modèle d'import	2		
	Annonces administratives	1		
	Ressources d'annonce admin	0.5		
	Rôles et permissions	0.5		
	Dashboard	0.5		
	Test unitaire	0.5		
	Test d'intégration	0.5		

	❖ **Partie Professeur**	**21**	**6.3**	Norhane + alae
	+ Ajouter	**5.5**		
	Séance	0.5		
	Fichier d'une séance	0.5		
	Examen	0.5		
	Corrigé d'un examen	0.5		
	Annonce prof	0.5		
	Ressource d'une annonce	0.5		
	Notes et résultats	1		
	Test unitaire	1		
	+ Modifier	**15**		
	Séance	1		
	Fichier d'une séance	1		
	Examen	1		
	Corrigé d'un examen	1		
	Annonce prof	1		
	Ressource d'une annonce	1		
	Notes et résultats	1		
	Test unitaire	2		
	+ Supprimer	**1**		
	Séance	1		
	Fichier d'une séance	1		
	Examen	0.5		
	Corrigé d'un examen	0.5		
	Annonce prof	1		
	Ressource d'une annonce	0.5		
	Notes et résultats	1		
	Test unitaire	1		
	+ Sélectionner	**21**		
	Tableau de bord professeur	2		
	Ses enseignements	1		

	Gérer les séances	0.5		
	Fichiers d'une séance	0.5		
	Gérer les examens	1		
	Corrigé d'un examen	0.5		
	Notes et résultats	3		
	Copie corrigée étudiant	2		
	Emploi du temps professeur	0.5		
	Prochaine séance	1		
	Annonces prof	2		
	Ressources d'une annonce	2		
	Dashboard	1		
	Test unitaire	1		
	Test d'intégration	1		
	❖ ***Partie Étudiant***	**21**	**2.1**	Norhane +aymane+Alae
	+ Ajouter	**1**		
	Justificatif d'absence	0.5		
	Test unitaire	0.5		
	+ Modifier	**1.5**		
	Profil	0.5		
	Justificatif d'absence	0.5		
	Test unitaire	0.5		
	+ Supprimer	**2**		
	Justificatif d'absence	1		
	Test unitaire	1		
	+ Sélectionner	**10**		
	Tableau de bord étudiant	2		
	Emploi du temps étudiant	1		
	Prochaine séance	0.5		
	Espace pédagogique	0.5		
	Présences étudiant	1		
	Justificatif d'absence	0.5		

	Relevé de notes / résultats	3		
	Consulter annonces	2		
	Ressources d'une annonce	0.5		
	Gérer son profil	1		
	Gérer ses notifications	2		
	Dashboard	2		
	Test unitaire	1		
	Test d'intégration	1		
	❖ **Notifications et fonctions communes**	**7**	**2.1**	Norhane + Aymane
	+ Ajouter	**3.5**		
	Notification	3		
	Test unitaire	0.5		
	+ Supprimer	**1**		
	Notification	0.5		
	Test unitaire	0.5		
	+ Sélectionner	**2.5**		
	Liste des notifications	0.5		
	Détail d'une notification	0.5		
	Notifications non lues	0.5		
	Test unitaire	0.5		
	Test d'intégration	0.5		
	❖ **Partie Sécurité**	**8.5**	**2.55**	Aymane+norhane + alae
	Gestion des rôles et permissions	2		
	Contrôle d'accès par rôle	1		
	Protection JWT	0.5		
	Validation des fichiers uploadés	1		
	Sécurisation des téléchargements	1		

	recaptcha	1		
	Test unitaire	1		
	Test d'intégration	1		
	❖ **Documentation**	**10**	**3**	
	Recette			
Mise en œuvre 7h	❖ **Test de validation**	**7**	**2.1**	Norhane + Aymane + Alae
Total		**309.5h = 12.89j**	**92.85h = 3.87j**	

Annexe 8 : Estimation des tâches

6 Lien GitHub du projet :

Voici le lien GitHub vers notre projet :

https://github.com/M-Aymane-404/myCampus.git